

Actividad - Proyecto práctico

<https://github.com/osoria80/08MIAR-Proyecto-Programacion>

La actividad se desarrollará en grupos pre-definidos de 4 alumnos. Se debe indicar los nombres en orden alfabético (de apellidos). Recordad que esta actividad se corresponde con un 30% de la nota final de la asignatura. Se debe entregar el trabajo en la presente notebook.

- Alumno 1: German Ricardo Malnero
- Alumno 2: Guillermo de Lecea Ramos
- Alumno 3: Oscar Soria Corral
- Alumno 4: Rigel Chuliá Ortega

PARTE 1 - Instalación y requisitos previos

Las prácticas han sido preparadas para poder realizarse en el entorno de trabajo de Google Colab. Sin embargo, esta plataforma presenta ciertas incompatibilidades a la hora de visualizar la renderización en gym. Por ello, para obtener estas visualizaciones, se deberá trasladar el entorno de trabajo a local. Por ello, el presente dossier presenta instrucciones para poder trabajar en ambos entornos. Siga los siguientes pasos para un correcto funcionamiento:

1. **LOCAL:** Preparar el enviroment, siguiendo las intrucciones detalladas en la sección *1.1.Preparar enviroment*.
2. **AMBOS:** Modificar las variables "mount" y "drive_mount" a la carpeta de trabajo en drive en el caso de estar en Colab, y ejecturar la celda *1.2.Localizar entorno de trabajo*.
3. **COLAB:** se deberá ejecutar las celdas correspondientes al montaje de la carpeta de trabajo en Drive. Esta corresponde a la sección *1.3.Montar carpeta de datos local*.
4. **AMBOS:** Instalar las librerías necesarias, siguiendo la sección *1.4.Instalar librerías necesarias*.

```
In [ ]: # Instalación de Librerías necesarias
```

```
%pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
%pip install "gymnasium[atari,accept-rom-license]"
%pip install ale-py
%pip install numpy pillow matplotlib
%pip install tqdm
```

Looking in indexes: <https://download.pytorch.org/whl/cu121>

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.11.0+cu128)

Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.26.0+cu128)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.4)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)

Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.1)

Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)

Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch) (9.19.0.56)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch) (2.28.9)

Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)

Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)

Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.4.1)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.90)

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (11.3.3.83)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (1.13.1.3)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.90)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (11.7.3.90)

Requirement already satisfied: nvidia-cusparsen-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.5.8.93)

2.5.8.93)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvidia-nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.93)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvidia-nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.93)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvidia-nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.90)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4->torch) (1.5.5)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: gymnasium[accept-rom-license,atari] in /usr/local/lib/python3.12/dist-packages (1.3.0)

WARNING: gymnasium 1.3.0 does not provide the extra 'accept-rom-license'

Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium[accept-rom-license,atari]) (2.0.2)

Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium[accept-rom-license,atari]) (3.1.2)

Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium[accept-rom-license,atari]) (4.15.0)

Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from gymnasium[accept-rom-license,atari]) (0.0.6)

Requirement already satisfied: ale_py>=0.9 in /usr/local/lib/python3.12/dist-packages (from gymnasium[accept-rom-license,atari]) (0.12.0)

Requirement already satisfied: ale-py in /usr/local/lib/python3.12/dist-packages (0.12.0)

Requirement already satisfied: numpy>1.20 in /usr/local/lib/python3.12/dist-packages (from ale-py) (2.0.2)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)

Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)

```
In [ ]: import torch

print("PyTorch version:", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU detectada:", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Usando dispositivo:", device)
```

PyTorch version: 2.11.0+cu128

CUDA disponible: True

GPU detectada: NVIDIA L4

Usando dispositivo: cuda

PARTE 2. Enunciado

Consideraciones a tener en cuenta:

- El entorno sobre el que trabajaremos será el indicado en el listado correspondien de cada grupo y el algoritmo que usaremos será *DQN*.
- Para nuestro ejercicio, el requisito mínimo será alcanzado cuando el agente consiga una **media de recompensa por encima de los puntos indicados en el listado por grupos en modo test**. Por ello, esta media de la recompensa se calculará a partir del código de test en la última celda del notebook.

Este proyecto práctico consta de tres partes:

1. [Implementar la red neuronal que se usará en la solución](#)
2. Implementar las distintas piezas de la solución DQN y probar al menos 3 propuestas diferentes de mejora.
3. Justificar la respuesta en relación a los resultados obtenidos e incluir al menos 3 gráficas relevantes comparando las 3 propuestas.

Rúbrica: Se valorará la originalidad en la solución aportada, así como la capacidad de discutir los resultados de forma detallada. El requisito mínimo servirá para aprobar la actividad, bajo premisa de que la discusión del resultado sera apropiada.

IMPORTANTE:

- Si no se consigue una puntuación óptima, responder sobre la mejor puntuación obtenida.
- Para entrenamientos largos, recordad que podéis usar checkpoints de vuestros modelos para retomar los entrenamientos. En este caso, recordad cambiar los parámetros adecuadamente (sobre todo los relacionados con el proceso de exploración).
- Se deberá entregar únicamente el notebook y los pesos del mejor modelo en un fichero .zip, de forma organizada.
- Cada alumno deberá de subir la solución de forma individual.

PARTE 3. Desarrollo

3.1 - Preparación previa

3.1.1 - Importar librerías

```
In [ ]: from __future__ import annotations

import os
import random
import time
from collections import deque, namedtuple

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from PIL import Image

# OPTIMIZACION GPU en COLAB
import torch.backends.cudnn as cudnn
cudnn.benchmark = True
#-----
```

```

import gymnasium as gym
import ale_py

import json
import pandas as pd

gym.register_envs(ale_py)

import matplotlib.pyplot as plt
import numpy as np

# Imprimimos por pantalla que esta configurado para entrenar CPU o GPU detectando por CUDA

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Dispositivo de entrenamiento:", device)

```

Dispositivo de entrenamiento: cuda

3.1.2 - Función para graficar resultados

Definimos la función para graficar y comparar resultados de los modelos.

```

In [ ]: def plot_training_history(history,
    rolling_window=3,
    figsize=(16, 10),
    save=False,
    filename="training_history.png"):

    fig, axes = plt.subplots(2, 3, figsize=figsize)
    axes = axes.ravel()

    x = history["step"]

    def moving_average(y, window):
        return (
            pd.Series(y)
            .rolling(window=window, min_periods=1)
            .mean()
            .to_numpy()

```

```
)

# =====
# 1. Reward
# =====
axes[0].plot(x, history["mean_reward_20ep"],
             linewidth=2,
             label="Reward")

if rolling_window > 1:
    axes[0].plot(
        x,
        moving_average(history["mean_reward_20ep"], rolling_window),
        linewidth=3,
        label="Media móvil"
    )

axes[0].set_title("Reward medio (últimos 20 episodios)")
axes[0].set_xlabel("Steps")
axes[0].set_ylabel("Reward")
axes[0].grid(True)
axes[0].legend()

# =====
# 2. Loss
# =====
axes[1].plot(x, history["loss"], linewidth=2)

if rolling_window > 1:
    axes[1].plot(
        x,
        moving_average(history["loss"], rolling_window),
        linewidth=3
    )

axes[1].set_title("Loss")
axes[1].set_xlabel("Steps")
axes[1].set_ylabel("Loss")
axes[1].grid(True)

# =====
```



```
# 3. Epsilon
# =====
axes[2].plot(x, history["epsilon"], linewidth=2)

axes[2].set_title("Epsilon")
axes[2].set_xlabel("Steps")
axes[2].set_ylabel("ε")
axes[2].grid(True)

# =====
# 4. Tiempo por intervalo
# =====
axes[3].plot(x, history["interval_time"], linewidth=2)

axes[3].set_title("Tiempo por intervalo")
axes[3].set_xlabel("Steps")
axes[3].set_ylabel("Segundos")
axes[3].grid(True)

# =====
# 5. Tiempo acumulado
# =====
tiempo_horas = np.array(history["total_time"]) / 3600

axes[4].plot(x, tiempo_horas, linewidth=2)

axes[4].set_title("Tiempo acumulado")
axes[4].set_xlabel("Steps")
axes[4].set_ylabel("Horas")
axes[4].grid(True)

# =====
# 6. COMPARATIVA: Reward vs Q-Valor (o Reward vs Loss por compatibilidad)
# =====
ax1 = axes[5]

# Dibujamos siempre el Reward de fondo en el eje Y izquierdo
ax1.plot(x,
        history["mean_reward_20ep"],
        label="Reward medio",
        color="tab:blue",
```

```
        linewidth=2)
ax1.set_ylabel("Reward medio", color="tab:blue")
ax1.tick_params(axis='y', labelcolor="tab:blue")

# Creamos el segundo eje Y derecho
ax2 = ax1.twinx()

# Si el historial tiene Q-valores, dibujamos la comparativa de Q-valores (DDQN vs DQN)
if "mean_q" in history and len(history["mean_q"]) > 0:
    ax2.plot(x,
             history["mean_q"],
             linestyle="--",
             linewidth=2,
             color="tab:orange",
             label="Q-Valor medio")

    # Opcional: añadimos una media móvil a los Q-valores para que se vea más limpia la tendencia
    if rolling_window > 1:
        ax2.plot(
            x,
            moving_average(history["mean_q"], rolling_window),
            linewidth=3,
            color="darkorange",
            label="Q-Valor móvil"
        )

    ax1.set_title("Evolución de Reward vs Q-Valor medio")
    ax2.set_ylabel("Q-Valor estimado", color="tab:orange")
    ax2.tick_params(axis='y', labelcolor="tab:orange")

# Si es un historial antiguo sin Q-valores, dejamos por defecto la comparativa de Loss
else:
    ax2.plot(x,
             history["loss"],
             linestyle="--",
             linewidth=2,
             color="tab:red",
             label="Loss")

    ax1.set_title("Evolución de Reward y Loss")
    ax2.set_ylabel("Loss", color="tab:red")
```

```
ax2.tick_params(axis='y', labelcolor="tab:red")

ax1.grid(True)

# Unificar leyendas de ambos ejes en un único cuadro
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper left")

plt.tight_layout()

if save:
    plt.savefig(filename,
                dpi=300,
                bbox_inches="tight")

plt.show()
```

3.1.3 - Configuración base

Definición de variables generales, entorno y de la clase Atari.

```
In [ ]: INPUT_SHAPE = (84, 84) # tamaño al que reescalamos los frames del juego
WINDOW_LENGTH = 4 # nº de frames apilados (ventana temporal) para enviar x imágenes juntas del juego

ENV_NAME = "ALE/SpaceInvaders-v5"

NB_STEPS_WARMUP = 20_000 # Pasos iniciales que permanecerá probando de forma aleatoria

SEED = 123 # definimos las semillas para que siempre se repitan los mismos experimentos
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

def make_env(render_mode=None):
    env = gym.make(ENV_NAME, render_mode=render_mode)
    return env

env = make_env()
```

Características del entorno

Extraemos información del juego para comprender el proceso.

```
In [ ]: # Información del juego dentro de env, para entender el proceso, Extraído directamente de la librería

print("Formato de la imagen (Observación):", env.observation_space.shape)
print("Tipo de datos de los píxeles:", env.observation_space.dtype)
print("Valor máximo y mínimo de un píxel:", env.observation_space.low.min(), "a", env.observation_space.high.max())

significado_acciones = env.unwrapped.get_action_meanings()

N_ACTIONS=env.action_space.n

print(f"El entorno tiene {env.action_space.n} acciones definidas.")
print("\nListado y significado de cada acción:")
for i, significado in enumerate(significado_acciones):
    print(f"Acción {i}: {significado}")
```

Formato de la imagen (Observación): (210, 160, 3)
 Tipo de datos de los píxeles: uint8
 Valor máximo y mínimo de un píxel: 0 a 255
 El entorno tiene 6 acciones definidas.

Listado y significado de cada acción:

Acción 0: NOOP
 Acción 1: FIRE
 Acción 2: RIGHT
 Acción 3: LEFT
 Acción 4: RIGHTFIRE
 Acción 5: LEFTFIRE

Clase Atari

```
In [ ]: class AtariProcessor:
    """Equivalente al Processor de keras-rl2: preprocesa observaciones, recompensas
    y mantiene una ventana (stack) de los últimos WINDOW_LENGTH frames procesados."""

    def __init__(self, input_shape=INPUT_SHAPE, window_length=WINDOW_LENGTH): # Configura el tamaño final de las imagenes y la
        self.input_shape = input_shape
        self.window_length = window_length
        self.frames = deque(maxlen=window_length) # Creamos deque por que es mas comodo para ir añadiendo el frame siguiente,

    def process_observation(self, observation): # Cogemos la imagen del juego, la dimensionamos a 84x84 y la convertimos a escala
        assert observation.ndim == 3
        img = Image.fromarray(observation)
        img = img.resize(self.input_shape).convert("L") # escala de grises 84x84 (libreria PIL), En el Space invader los colores
        return np.array(img, dtype=np.uint8)

    def process_reward(self, reward): # Normalizamos las recompensas en el rango -1 y 1. Si haya mucha diferencia en las recompensas
        return float(np.clip(reward, -1.0, 1.0))

    def reset(self, observation): # Configuración de la imagen inicial, 4 imágenes iguales por que no hay imágenes pasadas.
        frame = self.process_observation(observation)
        self.frames.clear()
        for _ in range(self.window_length):
            self.frames.append(frame)
        return self.get_state()
```

```

def step(self, observation): # Se ejecutara en cada fotograma si el agente esta jugando. coge la imagen despues de realiza
    frame = self.process_observation(observation)
    self.frames.append(frame)
    return self.get_state()

def get_state(self):
    #stacked = np.stack(self.frames, axis=0).astype(np.float32) / 255.0 # Normalizamos para que tenga valores entre 0 y 1
    stacked = np.stack(self.frames, axis=0)
    return stacked

```

3.2 - DQN

3.2.1 - Definición de la red neuronal DQN

```

In [ ]: class DQNCNN(nn.Module):
    """Red convolucional equivalente a la del notebook original (TF), pero en PyTorch.
    Entrada: (batch, WINDOW_LENGTH, 84, 84) -> usamos directamente NCHW (formato nativo de PyTorch),
    así que no necesitamos el truco de Permute que se usaba en TF/NHWC."""

    def __init__(self, n_actions, window_length=WINDOW_LENGTH):
        super().__init__()
        self.conv1 = nn.Conv2d(window_length, 32, kernel_size=8, stride=4)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1)

        # Calculamos el tamaño de salida tras las convoluciones para 84x84 de entrada
        conv_out_size = self._get_conv_out((window_length, 84, 84))

        self.fc1 = nn.Linear(conv_out_size, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, n_actions)

    def _get_conv_out(self, shape):
        o = torch.zeros(1, *shape)
        o = self.conv3(self.conv2(self.conv1(o)))
        return int(np.prod(o.shape))

```

```
def forward(self, x):
    x = F.relu(self.conv1(x))
    x = F.relu(self.conv2(x))
    x = F.relu(self.conv3(x))
    x = x.view(x.size(0), -1)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    return self.fc3(x)
```

```
model = DQNCNN(N_ACTIONS, WINDOW_LENGTH).to(device)
print(model)
```

```
n_params = sum(p.numel() for p in model.parameters())
print(f"Total de parámetros: {n_params:,}")
```

```
DQNCNN(
  (conv1): Conv2d(4, 32, kernel_size=(8, 8), stride=(4, 4))
  (conv2): Conv2d(32, 64, kernel_size=(4, 4), stride=(2, 2))
  (conv3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
  (fc1): Linear(in_features=3136, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=256, bias=True)
  (fc3): Linear(in_features=256, out_features=6, bias=True)
)
```

Total de parámetros: 1,816,998

3.2.2 - Implementación de DQN

```
In [ ]: Transition = namedtuple("Transition", ["state", "action", "reward", "next_state", "done"]) # (s,a,r,s)
```

Memoria

```
In [ ]: # Replay MEMORY para evitar que la RAM se colapse
```

```
"""
```

```
Su función es almacenar las experiencias del agente para poder
entrenar la red neuronal utilizando experiencias pasadas elegidas
de forma aleatoria.
```

Se utiliza un Buffer Circular (<https://www.baeldung.com/cs/circular-buffer?>)

"""

class ReplayMemory:

"""

Versión optimizada de la memoria de replay.

Pre-asigna espacio en memoria y realiza un muestreo vectorizado ultra-rápido, evitando crear objetos Transition intermedios.

"""

def __init__(self, capacity=1_000_000, state_shape=(4, 84, 84), device="cpu"): *# Inicializacion de variables*

"""

capacity: número máximo de experiencias que se pueden almacenar.

state_shape: tamaño de cada estado (4 imágenes de 84x84).

device: dispositivo donde posteriormente se utilizarán los datos.

"""

self.capacity = capacity

self.device = device

Se reserva toda la memoria desde el principio para mejorar el rendimiento.

Así no es necesario crear nuevos arrays continuamente.

Estados actuales

self.states = np.zeros((capacity, *state_shape), dtype=np.uint8)

Estados siguientes

self.next_states = np.zeros((capacity, *state_shape), dtype=np.uint8)

Acción realizada por el agente

self.actions = np.zeros(capacity, dtype=np.int64)

Recompensa obtenida

self.rewards = np.zeros(capacity, dtype=np.float32)

Indica si el episodio terminó (1=True, 0=False)

self.dones = np.zeros(capacity, dtype=np.float32)

Posición donde se guardará la siguiente experiencia

self.position = 0

Número de experiencias almacenadas actualmente

self.size = 0

def push(self, state, action, reward, next_state, done):

"""


```
Guarda una nueva experiencia en la memoria.

Si la memoria está llena, sobrescribe la experiencia más antigua.
"""

# Guardar todos los datos de la experiencia
self.states[self.position] = state
self.actions[self.position] = action
self.rewards[self.position] = reward
self.next_states[self.position] = next_state
self.dones[self.position] = done

# Avanzar una posición.
# Cuando llega al final vuelve al principio (buffer circular).
self.position = (self.position + 1) % self.capacity
self.size = min(self.size + 1, self.capacity)

def sample(self, batch_size):
    """
    Devuelve un lote de experiencias elegidas aleatoriamente.

    Este lote será utilizado para entrenar la red neuronal.
    """
    # Seleccionar índices aleatorios
    indices = np.random.randint(0, self.size, size=batch_size)

    # Obtener las experiencias correspondientes
    states = self.states[indices]
    actions = self.actions[indices]
    rewards = self.rewards[indices]
    next_states = self.next_states[indices]
    dones = self.dones[indices]

    # Se devuelven como arrays de NumPy.
    # Más adelante el agente los convertirá en tensores de PyTorch.
    return states, actions, rewards, next_states, dones

def __len__(self):
    """
    Devuelve cuántas experiencias hay almacenadas actualmente.
    Permite usar:
```

```

        len(memory)
    """
    return self.size

```

Política

```

In [ ]: # Política para modificar el parametro Epsilon y que el agente cambie el Exploracion por Explotación.
# value_max y value_min definen los valores de epsilon
# value_min indica que aunque sea un experto siempre deja algo a la exploración. =,1= 10%
# value_test es el porcentaje que usará durante el test.
# nb_steps número de pasos que tardará en descender hasta el mínimo seleccionado.

class LinearAnnealedEpsGreedy:

    def __init__(self, value_max=1.0, value_min=0.1, value_test=0.05, nb_steps=1_000_000): # Al crearlo inicializamos las variables
        self.value_max = value_max
        self.value_min = value_min
        self.value_test = value_test
        self.nb_steps = nb_steps

    def get_eps(self, step, training=True): # Devuelve el epsilon del entrenamiento en función de los pasos ejecutados y de si es entrenamiento
        if not training:
            return self.value_test
        fraction = min(1.0, step / float(self.nb_steps))
        return self.value_max + fraction * (self.value_min - self.value_max)

    def select_action(self, q_values, step, training=True): # Aquí es donde elige si exploramos o Explotamos. a través de un 1
        eps = self.get_eps(step, training=training)
        if random.random() < eps:
            return random.randrange(len(q_values))
        return int(np.argmax(q_values))

```

Agente

```

In [ ]: # Definimos el Agente para DQN

class DQNAgent:

    def __init__( # El constructor del agente

```

```

self,
model,
n_actions,
memory,
policy,
device,
gamma=0.99,
lr=2.5e-4,
nb_steps_warmup=NB_STEPS_WARMUP,
target_model_update=10_000,
train_interval=4,
batch_size=32,
): # Definicion del Agente y sus variables
    self.device = device
    self.model = model.to(device)
    self.target_model = DQNCNN(n_actions, WINDOW_LENGTH).to(device) # Indicamos la CNN que hara de agente.
    self.target_model.load_state_dict(self.model.state_dict())
    self.target_model.eval()

    self.n_actions = n_actions
    self.memory = memory
    self.policy = policy
    self.gamma = gamma
    self.nb_steps_warmup = nb_steps_warmup
    self.target_model_update = target_model_update
    self.train_interval = train_interval
    self.batch_size = batch_size

    self.optimizer = optim.Adam(self.model.parameters(), lr=lr)
    self.loss_fn = nn.SmoothL1Loss() # equivalente robusto a MAE/Huber usado en DQN

    self.step_count = 0

    @torch.no_grad() # Le pedimos a PyTorch que no guarde el historial del gradiente para liberar RAM. Acupa menos espacio de

    def act(self, state, training=True): #
        #state_t = torch.as_tensor(state, dtype=torch.float32, device=self.device).unsqueeze(0)
        state_t = (torch.as_tensor(state, dtype=torch.float32, device=self.device).unsqueeze(0) / 255.0)
        q_values = self.model(state_t).cpu().numpy()[0]
        return self.policy.select_action(q_values, self.step_count, training=training)

```

```

def remember(self, state, action, reward, next_state, done): #
    self.memory.push(state, action, reward, next_state, done)

def update_target(self):
    self.target_model.load_state_dict(self.model.state_dict())

def train_step(self):
    if len(self.memory) < self.batch_size:
        return None

    states, actions, rewards, next_states, dones = self.memory.sample(self.batch_size)

    #states = torch.as_tensor(states, dtype=torch.float32, device=self.device) Bug
    states = (torch.as_tensor(states, dtype=torch.float32, device=self.device) / 255.0)
    actions = torch.as_tensor(actions, dtype=torch.int64, device=self.device)
    rewards = torch.as_tensor(rewards, dtype=torch.float32, device=self.device)

    next_states = (torch.as_tensor(next_states, dtype=torch.float32, device=self.device) / 255.0)
    #next_states = torch.as_tensor(next_states, dtype=torch.float32, device=self.device) Bug
    dones = torch.as_tensor(dones, dtype=torch.float32, device=self.device)

    q_values = self.model(states).gather(1, actions.unsqueeze(1)).squeeze(1)

    with torch.no_grad():
        next_q_values = self.target_model(next_states).max(1)[0]
        target = rewards + self.gamma * next_q_values * (1.0 - dones) # Ecuacion de Bellman

    loss = self.loss_fn(q_values, target)

    self.optimizer.zero_grad()
    loss.backward()
    for p in self.model.parameters():
        if p.grad is not None:
            p.grad.data.clamp_(-1, 1)
    self.optimizer.step()

    mean_q = q_values.mean().item() # Promedio de los Q-valores predichos en el batch

    return loss.item(), mean_q

def save_weights(self, path):

```

```

torch.save(self.model.state_dict(), path)

def load_weights(self, path):
    state_dict = torch.load(path, map_location=self.device)
    self.model.load_state_dict(state_dict)
    self.target_model.load_state_dict(state_dict)

```

3.2.3 - Entrenamiento DQN

Configuración del entrenamiento

```

In [ ]: # --- Elige el modo de entrenamiento ---

NB_STEPS = 600000

weights_filename = f"dqn_{ENV_NAME.replace('/', '_')}weights.pt"
history_filename = f"history_dqn_SpaceInvaders.json"

print(f"Pasos de entrenamiento: {NB_STEPS:,}")

```

Pasos de entrenamiento: 600,000

```

In [ ]: from tqdm.auto import tqdm # Libreria para la barra de progreso

# Esta es la funcion general que lanzaremos cada vez que entrenemos un modelo

def train(env, agent, processor, nb_steps, log_interval=10_000):
    obs, info = env.reset(seed=SEED)
    state = processor.reset(obs)

    episode_reward = 0.0
    episode_steps = 0
    episode_count = 0
    recent_rewards = deque(maxlen=20)

    start_time = time.time()
    interval_start_time = start_time
    interval_loss = []

```

```
interval_reward = 0.0
accumulated_q_values = []

history = {
    "step": [],
    "mean_reward_20ep": [],
    "loss": [],
    "epsilon": [],
    "mean_q": [],
    "interval_time": [],
    "total_time": [],
}

n_intervals = nb_steps // log_interval
pbar = tqdm(total=log_interval, desc=f"Intervalo 1/{n_intervals}", unit="step")

for step in range(1, nb_steps + 1):
    agent.step_count = step

    if step <= agent.nb_steps_warmup:
        action = env.action_space.sample()
    else:
        action = agent.act(state, training=True)

    obs, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated

    proc_reward = processor.process_reward(reward)
    next_state = processor.step(obs)

    agent.remember(state, action, proc_reward, next_state, float(done))

    state = next_state
    episode_reward += reward
    episode_steps += 1
    interval_reward += proc_reward

    if step > agent.nb_steps_warmup and step % agent.train_interval == 0:
        loss, q_val = agent.train_step()
```

```

    if q_val is not None:
        accumulated_q_values.append(q_val)

    if loss is not None:
        interval_loss.append(loss)

if step > agent.nb_steps_warmup and step % agent.target_model_update == 0:
    agent.update_target()

if done:
    episode_count += 1
    recent_rewards.append(episode_reward)
    obs, info = env.reset()
    state = processor.reset(obs)
    episode_reward = 0.0
    episode_steps = 0

# --- Actualizamos la barra de progreso en cada step ---
mean_r_live = np.mean(recent_rewards) if recent_rewards else 0.0

current_loss = interval_loss[-1] if interval_loss else 0.0
current_q = accumulated_q_values[-1] if accumulated_q_values else 0.0

pbar.set_postfix({
    "reward": f"{mean_r_live:.1f}",
    "loss": f"{current_loss:.4f}",
    "mean_q": f"{current_q:.3f}",
    "eps": f"{agent.policy.get_eps(step, training=True):.3f}",
})
pbar.update(1)

if step % log_interval == 0:
    now = time.time()
    elapsed_total = now - start_time
    elapsed_interval = now - interval_start_time
    mean_r = np.mean(recent_rewards) if recent_rewards else 0.0
    mean_loss = np.mean(interval_loss) if interval_loss else 0.0
    eps = agent.policy.get_eps(step, training=True)

    history["step"].append(step)
    history["mean_reward_20ep"].append(mean_r)

```

```

history["loss"].append(mean_loss)
history["epsilon"].append(eps)
history["interval_time"].append(elapsed_interval)
history["total_time"].append(elapsed_total)
history["mean_q"].append(np.mean(accumulated_q_values))

# Cerramos la barra del intervalo actual y mostramos el resumen
pbar.close()
print(
    f"Step {step:>8,}/{nb_steps:,} | "
    f"reward(últ.20 ep): {mean_r:6.2f} | "
    f"loss: {mean_loss:7.4f} | "
    f"eps: {eps:5.3f} | "
    f"tiempo intervalo: {elapsed_interval:6.1f}s | "
    f"tiempo total: {elapsed_total:7.1f}s"
)
interval_loss = []
accumulated_q_values = []
interval_start_time = now

# Abrimos la barra para el siguiente intervalo (si queda alguno)
if step < nb_steps:
    current_interval = step // log_interval + 1
    pbar = tqdm(total=log_interval, desc=f"Intervalo {current_interval}/{n_intervals}", unit="step")

total_time = time.time() - start_time
print(f"\nEntrenamiento finalizado. Tiempo total: {total_time:.1f}s "
      f"({total_time/60:.2f} min, {total_time/3600:.2f} h)")

return agent, history

```

Instanciación del agente DQN

```

In [ ]: processor = AtariProcessor(INPUT_SHAPE, WINDOW_LENGTH)

memory = ReplayMemory(capacity=1_000_000)

policy = LinearAnnealedEpsGreedy( # Configuramos la Policy del agente para el entrenamiento
    value_max=1.0,
    value_min=0.1,

```



```

    value_test=0.05,
    nb_steps=NB_STEPS, # Se adapta al numero de pasos que tengamos en el entrenamiento
)

agent = DQNAgent(
    model=model,
    n_actions=N_ACTIONS,
    memory=memory,
    policy=policy,
    device=device,
    gamma=0.99,
    lr=2.5e-4,
    nb_steps_warmup=NB_STEPS_WARMUP, # Numero de pasos que o fotogramas que el juego siempre jugara de forma aleatoria
    target_model_update=10_000,
    train_interval=4,
    batch_size=32,
)

```

Entrenamiento

```

In [ ]: agent, history_dqn = train(
    env,
    agent,
    processor,
    nb_steps=NB_STEPS,
)

agent.save_weights(weights_filename)
print("Pesos guardados en:", weights_filename)

# Guardamos también el histórico de métricas

with open(history_filename, "w", encoding="utf-8") as f:
    json.dump(history_dqn, f)
print("Histórico de entrenamiento guardado en:", history_filename)

```

Intervalo 1/60: 100%  10000/10000 [00:19<00:00, 516.75step/s, reward=152.4, loss=0.0000, mean_q=0.000, eps=0.985]

Step 10,000/600,000 | reward(últ.20 ep): 152.35 | loss: 0.0000 | eps: 0.985 | tiempo intervalo: 19.8s | tiempo total: 19.8s

Intervalo 2/60: 100%  10000/10000 [00:19<00:00, 505.86step/s, reward=140.2, loss=0.0000, mean_q=0.000, eps=0.970]

Step 20,000/600,000 | reward(últ.20 ep): 140.25 | loss: 0.0000 | eps: 0.970 | tiempo intervalo: 19.7s | tiempo total: 39.6s

Intervalo 3/60: 100%  10000/10000 [00:45<00:00, 215.10step/s, reward=134.0, loss=0.0024, mean_q=0.223, eps=0.955]

Step 30,000/600,000 | reward(últ.20 ep): 134.00 | loss: 0.0069 | eps: 0.955 | tiempo intervalo: 46.0s | tiempo total: 85.6s

Intervalo 4/60: 100%  10000/10000 [00:46<00:00, 222.88step/s, reward=166.8, loss=0.0009, mean_q=0.196, eps=0.940]

Step 40,000/600,000 | reward(últ.20 ep): 166.75 | loss: 0.0065 | eps: 0.940 | tiempo intervalo: 46.1s | tiempo total: 131.7s

Intervalo 5/60: 100%  10000/10000 [00:47<00:00, 208.51step/s, reward=118.0, loss=0.0230, mean_q=0.327, eps=0.925]

Step 50,000/600,000 | reward(últ.20 ep): 118.00 | loss: 0.0070 | eps: 0.925 | tiempo intervalo: 47.8s | tiempo total: 179.5s

Intervalo 6/60: 100%  10000/10000 [00:47<00:00, 221.62step/s, reward=110.8, loss=0.0053, mean_q=0.244, eps=0.910]

Step 60,000/600,000 | reward(últ.20 ep): 110.75 | loss: 0.0066 | eps: 0.910 | tiempo intervalo: 47.4s | tiempo total: 226.9s

Intervalo 7/60: 100%  10000/10000 [00:46<00:00, 227.58step/s, reward=161.8, loss=0.0052, mean_q=0.275, eps=0.895]

Step 70,000/600,000 | reward(últ.20 ep): 161.75 | loss: 0.0073 | eps: 0.895 | tiempo intervalo: 46.0s | tiempo total: 272.9s

Intervalo 8/60: 100%  10000/10000 [00:45<00:00, 222.05step/s, reward=130.8, loss=0.0074, mean_q=0.495, eps=0.880]

Step 80,000/600,000 | reward(últ.20 ep): 130.75 | loss: 0.0073 | eps: 0.880 | tiempo intervalo: 45.9s | tiempo total: 318.9s

Intervalo 9/60: 100%  10000/10000 [00:45<00:00, 221.11step/s, reward=148.0, loss=0.0025, mean_q=0.343, eps=0.865]

Step 90,000/600,000 | reward(últ.20 ep): 148.00 | loss: 0.0076 | eps: 0.865 | tiempo intervalo: 45.8s | tiempo total: 364.7s

Intervalo 10/60: 100%  10000/10000 [00:46<00:00, 215.61step/s, reward=153.8, loss=0.0014, mean_q=0.414, eps=0.850]

Step 100,000/600,000 | reward(últ.20 ep): 153.75 | loss: 0.0085 | eps: 0.850 | tiempo intervalo: 46.1s | tiempo total: 410.7s

Intervalo 11/60: 100%  10000/10000 [00:45<00:00, 214.87step/s, reward=208.5, loss=0.0213, mean_q=0.401, eps=0.835]

Step 110,000/600,000 | reward(últ.20 ep): 208.50 | loss: 0.0089 | eps: 0.835 | tiempo intervalo: 45.8s | tiempo total: 456.5s

Intervalo 12/60: 100%  10000/10000 [00:45<00:00, 211.83step/s, reward=208.0, loss=0.0112, mean_q=0.453, eps=0.820]
Step 120,000/600,000 | reward(últ.20 ep): 208.00 | loss: 0.0085 | eps: 0.820 | tiempo intervalo: 45.8s | tiempo total: 502.4s

Intervalo 13/60: 100%  10000/10000 [00:46<00:00, 218.23step/s, reward=199.8, loss=0.0040, mean_q=0.447, eps=0.805]
Step 130,000/600,000 | reward(últ.20 ep): 199.75 | loss: 0.0090 | eps: 0.805 | tiempo intervalo: 46.3s | tiempo total: 548.7s

Intervalo 14/60: 100%  10000/10000 [00:46<00:00, 214.18step/s, reward=152.8, loss=0.0118, mean_q=0.622, eps=0.790]
Step 140,000/600,000 | reward(últ.20 ep): 152.75 | loss: 0.0089 | eps: 0.790 | tiempo intervalo: 46.1s | tiempo total: 594.8s

Intervalo 15/60: 100%  10000/10000 [00:47<00:00, 221.36step/s, reward=200.5, loss=0.0334, mean_q=0.685, eps=0.775]
Step 150,000/600,000 | reward(últ.20 ep): 200.50 | loss: 0.0100 | eps: 0.775 | tiempo intervalo: 47.7s | tiempo total: 642.5s

Intervalo 16/60: 100%  10000/10000 [00:46<00:00, 214.22step/s, reward=168.8, loss=0.0075, mean_q=0.592, eps=0.760]
Step 160,000/600,000 | reward(últ.20 ep): 168.75 | loss: 0.0103 | eps: 0.760 | tiempo intervalo: 46.3s | tiempo total: 688.8s

Intervalo 17/60: 100%  10000/10000 [00:46<00:00, 216.40step/s, reward=194.0, loss=0.0046, mean_q=0.795, eps=0.745]
Step 170,000/600,000 | reward(últ.20 ep): 194.00 | loss: 0.0099 | eps: 0.745 | tiempo intervalo: 46.6s | tiempo total: 735.4s

Intervalo 18/60: 100%  10000/10000 [00:46<00:00, 221.49step/s, reward=216.2, loss=0.0147, mean_q=0.872, eps=0.730]
Step 180,000/600,000 | reward(últ.20 ep): 216.25 | loss: 0.0105 | eps: 0.730 | tiempo intervalo: 46.5s | tiempo total: 781.9s

Intervalo 19/60: 100%  10000/10000 [00:46<00:00, 217.02step/s, reward=195.8, loss=0.0044, mean_q=0.745, eps=0.715]
Step 190,000/600,000 | reward(últ.20 ep): 195.75 | loss: 0.0106 | eps: 0.715 | tiempo intervalo: 46.6s | tiempo total: 828.5s

Intervalo 20/60: 100%  10000/10000 [00:46<00:00, 213.06step/s, reward=226.5, loss=0.0114, mean_q=0.796, eps=0.700]
Step 200,000/600,000 | reward(últ.20 ep): 226.50 | loss: 0.0108 | eps: 0.700 | tiempo intervalo: 46.5s | tiempo total: 874.9s

Intervalo 21/60: 100%  10000/10000 [00:46<00:00, 217.48step/s, reward=248.0, loss=0.0036, mean_q=0.875, eps=0.685]
Step 210,000/600,000 | reward(últ.20 ep): 248.00 | loss: 0.0107 | eps: 0.685 | tiempo intervalo: 46.6s | tiempo total: 921.6s

Intervalo 22/60: 100%  10000/10000 [00:46<00:00, 216.59step/s, reward=226.0, loss=0.0285, mean_q=1.074, eps=0.670]

Step 220,000/600,000 | reward(últ.20 ep): 226.00 | loss: 0.0118 | eps: 0.670 | tiempo intervalo: 46.7s | tiempo total: 96 8.2s

Intervalo 23/60: 100%  10000/10000 [00:46<00:00, 214.08step/s, reward=280.5, loss=0.0092, mean_q=1.213, eps=0.655]

Step 230,000/600,000 | reward(últ.20 ep): 280.50 | loss: 0.0119 | eps: 0.655 | tiempo intervalo: 46.7s | tiempo total: 101 4.9s

Intervalo 24/60: 100%  10000/10000 [00:46<00:00, 211.24step/s, reward=249.2, loss=0.0188, mean_q=0.889, eps=0.640]

Step 240,000/600,000 | reward(últ.20 ep): 249.25 | loss: 0.0130 | eps: 0.640 | tiempo intervalo: 46.7s | tiempo total: 106 1.6s

Intervalo 25/60: 100%  10000/10000 [00:46<00:00, 212.65step/s, reward=203.2, loss=0.0099, mean_q=1.243, eps=0.625]

Step 250,000/600,000 | reward(últ.20 ep): 203.25 | loss: 0.0130 | eps: 0.625 | tiempo intervalo: 46.5s | tiempo total: 110 8.1s

Intervalo 26/60: 100%  10000/10000 [00:46<00:00, 208.26step/s, reward=193.0, loss=0.0142, mean_q=1.179, eps=0.610]

Step 260,000/600,000 | reward(últ.20 ep): 193.00 | loss: 0.0125 | eps: 0.610 | tiempo intervalo: 46.9s | tiempo total: 115 5.0s

Intervalo 27/60: 100%  10000/10000 [00:47<00:00, 211.56step/s, reward=202.5, loss=0.0054, mean_q=1.043, eps=0.595]

Step 270,000/600,000 | reward(últ.20 ep): 202.50 | loss: 0.0127 | eps: 0.595 | tiempo intervalo: 47.3s | tiempo total: 120 2.3s

Intervalo 28/60: 100%  10000/10000 [00:47<00:00, 209.19step/s, reward=244.0, loss=0.0069, mean_q=1.173, eps=0.580]

Step 280,000/600,000 | reward(últ.20 ep): 244.00 | loss: 0.0126 | eps: 0.580 | tiempo intervalo: 47.3s | tiempo total: 124 9.6s

Intervalo 29/60: 100%  10000/10000 [00:47<00:00, 204.57step/s, reward=225.0, loss=0.0081, mean_q=1.184, eps=0.565]

Step 290,000/600,000 | reward(últ.20 ep): 225.00 | loss: 0.0129 | eps: 0.565 | tiempo intervalo: 47.3s | tiempo total: 129 6.9s

Intervalo 30/60: 100%  10000/10000 [00:46<00:00, 215.33step/s, reward=244.2, loss=0.0085, mean_q=1.411, eps=0.550]

Step 300,000/600,000 | reward(últ.20 ep): 244.25 | loss: 0.0132 | eps: 0.550 | tiempo intervalo: 47.0s | tiempo total: 134 3.9s

Intervalo 31/60: 100%  10000/10000 [00:46<00:00, 217.52step/s, reward=248.0, loss=0.0128, mean_q=1.483, eps=0.535]

Step 310,000/600,000 | reward(últ.20 ep): 248.00 | loss: 0.0143 | eps: 0.535 | tiempo intervalo: 46.7s | tiempo total: 139 0.5s

Intervalo 32/60: 100%  10000/10000 [00:46<00:00, 212.47step/s, reward=269.5, loss=0.0074, mean_q=1.340, eps=0.520]

Step 320,000/600,000 | reward(últ.20 ep): 269.50 | loss: 0.0149 | eps: 0.520 | tiempo intervalo: 46.7s | tiempo total: 143 7.2s

Intervalo 33/60: 100%  10000/10000 [00:46<00:00, 216.13step/s, reward=246.0, loss=0.0089, mean_q=1.530, eps=0.505]
Step 330,000/600,000 | reward(últ.20 ep): 246.00 | loss: 0.0144 | eps: 0.505 | tiempo intervalo: 46.5s | tiempo total: 148 3.7s

Intervalo 34/60: 100%  10000/10000 [00:46<00:00, 215.27step/s, reward=287.0, loss=0.0086, mean_q=1.748, eps=0.490]
Step 340,000/600,000 | reward(últ.20 ep): 287.00 | loss: 0.0146 | eps: 0.490 | tiempo intervalo: 46.4s | tiempo total: 153 0.2s

Intervalo 35/60: 100%  10000/10000 [00:46<00:00, 217.09step/s, reward=291.0, loss=0.0111, mean_q=1.588, eps=0.475]
Step 350,000/600,000 | reward(últ.20 ep): 291.00 | loss: 0.0152 | eps: 0.475 | tiempo intervalo: 46.8s | tiempo total: 157 6.9s

Intervalo 36/60: 100%  10000/10000 [00:47<00:00, 215.01step/s, reward=321.5, loss=0.0174, mean_q=1.828, eps=0.460]
Step 360,000/600,000 | reward(últ.20 ep): 321.50 | loss: 0.0153 | eps: 0.460 | tiempo intervalo: 47.5s | tiempo total: 162 4.4s

Intervalo 37/60: 100%  10000/10000 [00:46<00:00, 215.01step/s, reward=313.2, loss=0.0063, mean_q=1.990, eps=0.445]
Step 370,000/600,000 | reward(últ.20 ep): 313.25 | loss: 0.0163 | eps: 0.445 | tiempo intervalo: 46.6s | tiempo total: 167 1.0s

Intervalo 38/60: 100%  10000/10000 [00:46<00:00, 212.98step/s, reward=292.2, loss=0.0262, mean_q=1.657, eps=0.430]
Step 380,000/600,000 | reward(últ.20 ep): 292.25 | loss: 0.0162 | eps: 0.430 | tiempo intervalo: 46.9s | tiempo total: 171 7.9s

Intervalo 39/60: 100%  10000/10000 [00:46<00:00, 222.11step/s, reward=293.0, loss=0.0060, mean_q=1.700, eps=0.415]
Step 390,000/600,000 | reward(últ.20 ep): 293.00 | loss: 0.0165 | eps: 0.415 | tiempo intervalo: 46.7s | tiempo total: 176 4.6s

Intervalo 40/60: 100%  10000/10000 [00:46<00:00, 218.90step/s, reward=293.5, loss=0.0066, mean_q=1.676, eps=0.400]
Step 400,000/600,000 | reward(últ.20 ep): 293.50 | loss: 0.0171 | eps: 0.400 | tiempo intervalo: 46.5s | tiempo total: 181 1.0s

Intervalo 41/60: 100%  10000/10000 [00:46<00:00, 211.86step/s, reward=276.5, loss=0.0055, mean_q=2.106, eps=0.385]
Step 410,000/600,000 | reward(últ.20 ep): 276.50 | loss: 0.0167 | eps: 0.385 | tiempo intervalo: 46.7s | tiempo total: 185 7.7s

Intervalo 42/60: 100%  10000/10000 [00:46<00:00, 214.88step/s, reward=346.8, loss=0.0084, mean_q=1.962, eps=0.370]
Step 420,000/600,000 | reward(últ.20 ep): 346.75 | loss: 0.0168 | eps: 0.370 | tiempo intervalo: 46.5s | tiempo total: 190 4.3s

Intervalo 43/60: 100%  10000/10000 [00:47<00:00, 209.44step/s, reward=342.5, loss=0.0138, mean_q=2.150, eps=0.355]

Step 430,000/600,000 | reward(últ.20 ep): 342.50 | loss: 0.0167 | eps: 0.355 | tiempo intervalo: 47.7s | tiempo total: 195 2.0s

Intervalo 44/60: 100%  10000/10000 [00:47<00:00, 205.92step/s, reward=349.2, loss=0.0577, mean_q=1.665, eps=0.340]

Step 440,000/600,000 | reward(últ.20 ep): 349.25 | loss: 0.0179 | eps: 0.340 | tiempo intervalo: 47.9s | tiempo total: 199 9.9s

Intervalo 45/60: 100%  10000/10000 [00:47<00:00, 204.48step/s, reward=290.8, loss=0.0184, mean_q=2.074, eps=0.325]

Step 450,000/600,000 | reward(últ.20 ep): 290.75 | loss: 0.0184 | eps: 0.325 | tiempo intervalo: 47.2s | tiempo total: 204 7.2s

Intervalo 46/60: 100%  10000/10000 [00:47<00:00, 208.45step/s, reward=270.8, loss=0.0128, mean_q=2.153, eps=0.310]

Step 460,000/600,000 | reward(últ.20 ep): 270.75 | loss: 0.0189 | eps: 0.310 | tiempo intervalo: 47.4s | tiempo total: 209 4.5s

Intervalo 47/60: 100%  10000/10000 [00:47<00:00, 209.14step/s, reward=321.2, loss=0.0390, mean_q=2.177, eps=0.295]

Step 470,000/600,000 | reward(últ.20 ep): 321.25 | loss: 0.0184 | eps: 0.295 | tiempo intervalo: 47.0s | tiempo total: 214 1.5s

Intervalo 48/60: 100%  10000/10000 [00:47<00:00, 203.37step/s, reward=263.5, loss=0.0106, mean_q=2.141, eps=0.280]

Step 480,000/600,000 | reward(últ.20 ep): 263.50 | loss: 0.0188 | eps: 0.280 | tiempo intervalo: 47.2s | tiempo total: 218 8.8s

Intervalo 49/60: 100%  10000/10000 [00:48<00:00, 206.92step/s, reward=318.2, loss=0.0205, mean_q=2.299, eps=0.265]

Step 490,000/600,000 | reward(últ.20 ep): 318.25 | loss: 0.0197 | eps: 0.265 | tiempo intervalo: 48.1s | tiempo total: 223 6.9s

Intervalo 50/60: 100%  10000/10000 [00:47<00:00, 206.42step/s, reward=356.5, loss=0.0232, mean_q=2.432, eps=0.250]

Step 500,000/600,000 | reward(últ.20 ep): 356.50 | loss: 0.0191 | eps: 0.250 | tiempo intervalo: 47.3s | tiempo total: 228 4.2s

Intervalo 51/60: 100%  10000/10000 [00:47<00:00, 205.26step/s, reward=377.0, loss=0.0230, mean_q=2.435, eps=0.235]

Step 510,000/600,000 | reward(últ.20 ep): 377.00 | loss: 0.0197 | eps: 0.235 | tiempo intervalo: 47.3s | tiempo total: 233 1.5s

Intervalo 52/60: 100%  10000/10000 [00:46<00:00, 211.29step/s, reward=302.0, loss=0.0485, mean_q=2.415, eps=0.220]

Step 520,000/600,000 | reward(últ.20 ep): 302.00 | loss: 0.0197 | eps: 0.220 | tiempo intervalo: 47.0s | tiempo total: 237 8.5s

Intervalo 53/60: 100%  10000/10000 [00:47<00:00, 214.94step/s, reward=348.2, loss=0.0152, mean_q=2.721, eps=0.205]

Step 530,000/600,000 | reward(últ.20 ep): 348.25 | loss: 0.0208 | eps: 0.205 | tiempo intervalo: 47.1s | tiempo total: 242 5.6s

Intervalo 54/60: 100%  10000/10000 [00:47<00:00, 202.50step/s, reward=348.5, loss=0.0061, mean_q=2.287, eps=0.190]
 Step 540,000/600,000 | reward(últ.20 ep): 348.50 | loss: 0.0201 | eps: 0.190 | tiempo intervalo: 48.0s | tiempo total: 247 3.6s

Intervalo 55/60: 100%  10000/10000 [00:47<00:00, 203.76step/s, reward=348.0, loss=0.0453, mean_q=2.527, eps=0.175]
 Step 550,000/600,000 | reward(últ.20 ep): 348.00 | loss: 0.0203 | eps: 0.175 | tiempo intervalo: 47.7s | tiempo total: 252 1.3s

Intervalo 56/60: 100%  10000/10000 [00:47<00:00, 206.86step/s, reward=330.8, loss=0.0271, mean_q=2.604, eps=0.160]
 Step 560,000/600,000 | reward(últ.20 ep): 330.75 | loss: 0.0208 | eps: 0.160 | tiempo intervalo: 47.6s | tiempo total: 256 8.9s

Intervalo 57/60: 100%  10000/10000 [00:47<00:00, 205.50step/s, reward=354.8, loss=0.0567, mean_q=2.211, eps=0.145]
 Step 570,000/600,000 | reward(últ.20 ep): 354.75 | loss: 0.0212 | eps: 0.145 | tiempo intervalo: 47.4s | tiempo total: 261 6.3s

Intervalo 58/60: 100%  10000/10000 [00:47<00:00, 207.89step/s, reward=400.8, loss=0.0117, mean_q=3.185, eps=0.130]
 Step 580,000/600,000 | reward(últ.20 ep): 400.75 | loss: 0.0225 | eps: 0.130 | tiempo intervalo: 47.0s | tiempo total: 266 3.3s

Intervalo 59/60: 100%  10000/10000 [00:47<00:00, 207.37step/s, reward=385.5, loss=0.0547, mean_q=2.573, eps=0.115]
 Step 590,000/600,000 | reward(últ.20 ep): 385.50 | loss: 0.0226 | eps: 0.115 | tiempo intervalo: 47.2s | tiempo total: 271 0.5s

Intervalo 60/60: 100%  10000/10000 [00:47<00:00, 212.66step/s, reward=333.2, loss=0.0116, mean_q=2.899, eps=0.100]
 Step 600,000/600,000 | reward(últ.20 ep): 333.25 | loss: 0.0221 | eps: 0.100 | tiempo intervalo: 47.4s | tiempo total: 275 7.9s

Entrenamiento finalizado. Tiempo total: 2757.9s (45.97 min, 0.77 h)
 Pesos guardados en: dqn_ALE_SpaceInvaders-v5_weights.pt
 Histórico de entrenamiento guardado en: history_dqn_SpaceInvaders.json

3.2.4 - Test DQN

```
In [ ]: def test(agent, processor, env_name, n_episodes=10, render=False):
        test_env = make_env(render_mode="human" if render else None)
        rewards = []

        for ep in range(1, n_episodes + 1):
```

```

obs, info = test_env.reset()
state = processor.reset(obs)
done = False
total_reward = 0.0
steps = 0

while not done:
    action = agent.act(state, training=False)
    obs, reward, terminated, truncated, info = test_env.step(action)
    done = terminated or truncated
    state = processor.step(obs)
    total_reward += reward
    steps += 1

rewards.append(total_reward)
print(f"Episodio {ep}: reward: {total_reward:.3f}, steps: {steps}")

test_env.close()
mean_reward = float(np.mean(rewards))
print(f"\nRecompensa media en {n_episodes} episodios: {mean_reward:.3f}")
return rewards, mean_reward

agent.load_weights(weights_filename)
test_rewards, mean_reward = test(agent, processor, ENV_NAME, n_episodes=10, render=False)

```

```

Episodio 1: reward: 525.000, steps: 1005
Episodio 2: reward: 230.000, steps: 601
Episodio 3: reward: 540.000, steps: 737
Episodio 4: reward: 390.000, steps: 664
Episodio 5: reward: 545.000, steps: 1003
Episodio 6: reward: 265.000, steps: 574
Episodio 7: reward: 220.000, steps: 511
Episodio 8: reward: 310.000, steps: 451
Episodio 9: reward: 195.000, steps: 476
Episodio 10: reward: 145.000, steps: 336

```

```
Recompensa media en 10 episodios: 336.500
```

```
In [ ]: env.close()
```

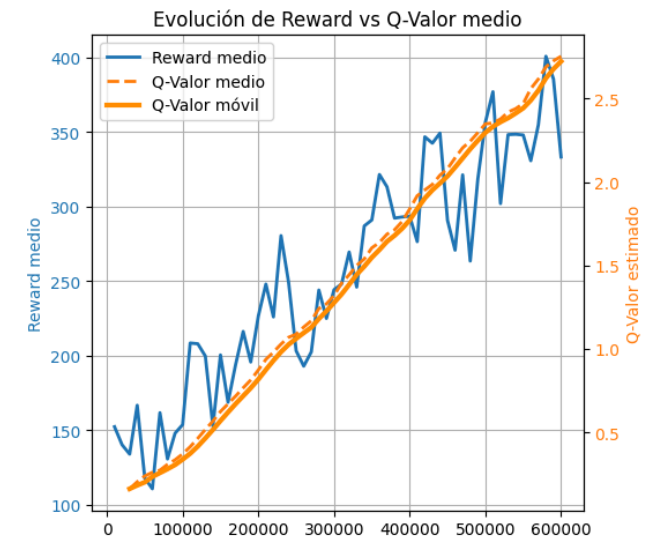
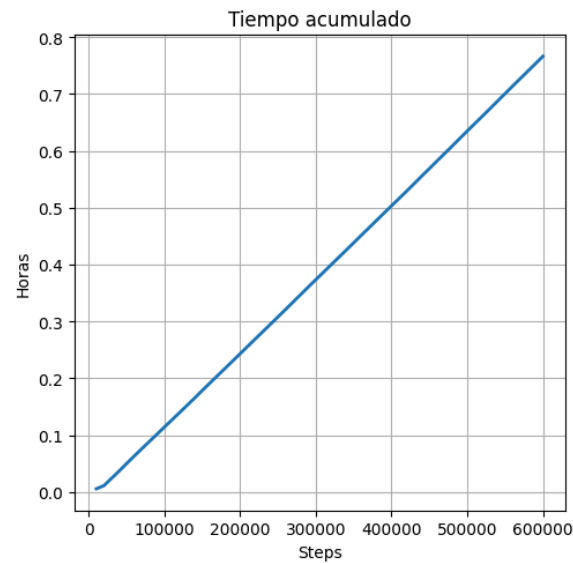
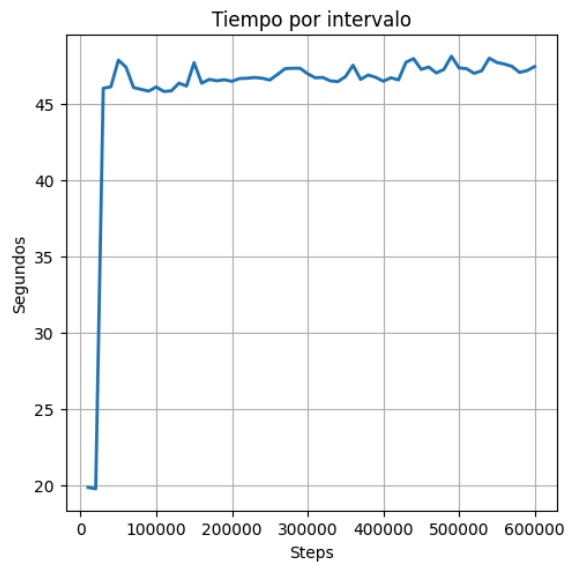
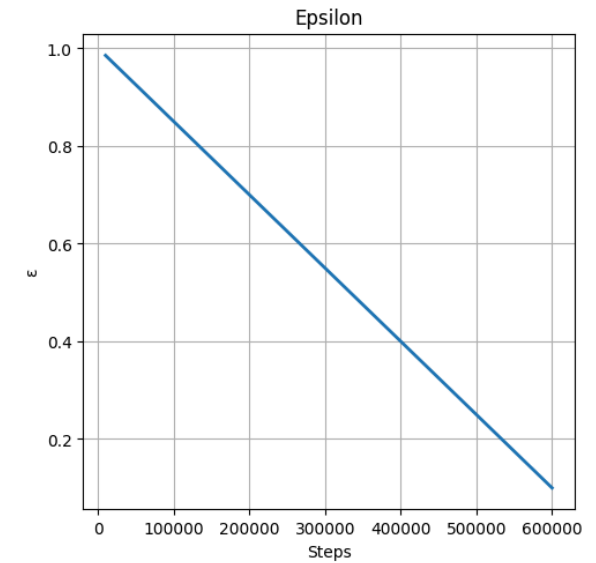
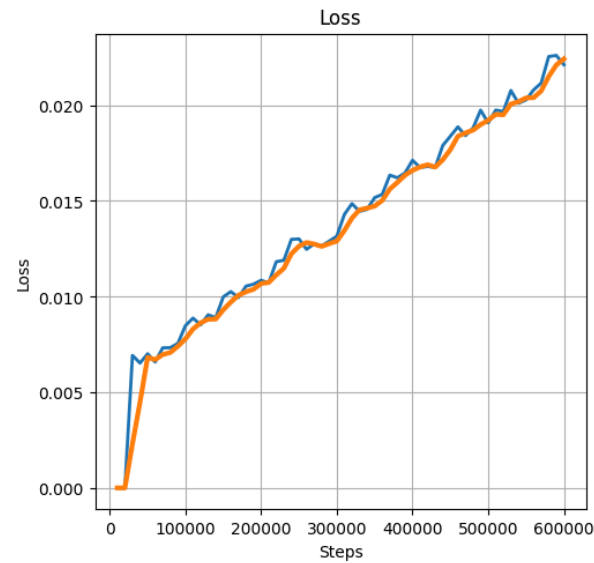
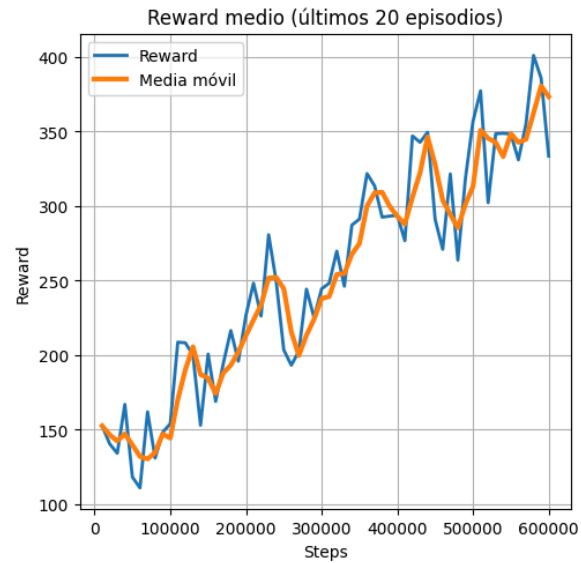

3.2.5 - Análisis de resultados

```
In [ ]: # Siempre traemos los datos del .json

history_filename = "history_dqn_SpaceInvaders.json"

with open(history_filename, "r", encoding="utf-8") as f:
    history = json.load(f)

plot_training_history(history)
```



3.3 - Implementación de 3 mejoras propuestas

3.3.1 - Double DQN (DDQN)

Uno de los principales problemas de DQN es que tiende a sobrevalorar determinadas acciones, especialmente en las primeras fases del entrenamiento dónde las estimaciones aún son bastante imprecisas. Esto se debe a que, en DQN, la misma red neuronal se utiliza tanto para seleccionar la mejor acción como para estimar el valor Q.

Para mitigar este efecto, se propone utilizar redes independientes para cada una de las tareas (estimar Q y elegir la acción), conocido como "Double DQN". En este caso, la red principal selecciona cuál es la mejor acción disponible, mientras que la red objetivo calcula el valor de esa acción. Al dividir los procesos de selección y evaluación, las estimaciones de Q son más realistas y el aprendizaje resulta más estable.

Definición

```
In [ ]: class DDQNAgent(DQNAgent):
    """
    Agente Double DQN (DDQN).
    Hereda de DQNAgent pero sobrescribe train_step() para desacoplar
    la selección de la mejor acción (Policy Net) y su evaluación (Target Net).
    """

    def train_step(self):
        # Comprobar si hay suficientes experiencias en memoria para entrenar
        if len(self.memory) < self.batch_size:
            return None

        # Extraer un lote (batch) aleatorio de la memoria
        states, actions, rewards, next_states, dones = self.memory.sample(self.batch_size)

        # Pasar los arrays a tensores en el dispositivo activo (GPU/CPU)
        # states = torch.as_tensor(states, dtype=torch.float32, device=self.device) bug
        states = torch.as_tensor(states, dtype=torch.float32, device=self.device)/255.0

        actions = torch.as_tensor(actions, dtype=torch.int64, device=self.device)
        rewards = torch.as_tensor(rewards, dtype=torch.float32, device=self.device)

        #next_states = torch.as_tensor(next_states, dtype=torch.float32, device=self.device)
        next_states = torch.as_tensor(next_states, dtype=torch.float32, device=self.device)/255.0
        dones = torch.as_tensor(dones, dtype=torch.float32, device=self.device)

        # Obtener predicciones actuales Q(s, a) de la red principal
        q_values = self.model(states).gather(1, actions.unsqueeze(1)).squeeze(1)
```

```

# Aqui est ala diferencia respecto a DQN normal
with torch.no_grad():
    # A. SELECCIÓN: La Red Principal (model) elige la mejor acción futura
    best_actions = self.model(next_states).argmax(dim=1).unsqueeze(1)

    # B. EVALUACIÓN: La Red Objetivo (target_model) evalúa el valor de esa acción específica
    next_q_values = self.target_model(next_states).gather(1, best_actions).squeeze(1)

    # C. Cálculo del Target de Bellman
    target = rewards + self.gamma * next_q_values * (1.0 - dones)
# -----

# Calcular la pérdida
loss = self.loss_fn(q_values, target)

self.optimizer.zero_grad()
loss.backward()

# Recorte de gradientes para asegurar estabilidad
for p in self.model.parameters():
    if p.grad is not None:
        p.grad.data.clamp_(-1, 1)

self.optimizer.step()

mean_q = q_values.mean().item() # Promedio de Los Q-Valores

return loss.item(), mean_q

```

```

In [ ]: # --- Elige el modo de entrenamiento ---
        # 60.000 para verificar que todo funciona)
        # 500.000 - 1.000.000 pasos (entrenamiento real)

        NB_STEPS = 600000

        weights_filename = f"ddqn_{ENV_NAME.replace('/', '_')}_weights.pt"
        history_filename = f"history_ddqn_SpaceInvaders.json"

        print(f"Pasos de entrenamiento: {NB_STEPS:,}")

```

Pasos de entrenamiento: 600,000

Instanciación y entreno

```
In [ ]: # Instanciamos las tres partes del algoritmo

processor = AtariProcessor(INPUT_SHAPE, WINDOW_LENGTH)
memory = ReplayMemory(capacity=1_000_000)

policy = LinearAnnealedEpsGreedy(
    value_max=1.0,
    value_min=0.1,
    value_test=0.05,
    nb_steps=NB_STEPS,
)

ddqn_model = DQNCNN(N_ACTIONS, WINDOW_LENGTH).to(device)

agent = DDQNAgent(
    model=ddqn_model,
    n_actions=N_ACTIONS,
    memory=memory,
    policy=policy,
    device=device,
    gamma=0.99,
    lr=2.5e-4,
    nb_steps_warmup=NB_STEPS_WARMUP,
    target_model_update=10_000,
    train_interval=4,
    batch_size=32,
)

agent, history_ddqn = train(
    env,
    agent,
    processor,
    nb_steps=NB_STEPS,
)
```

```

agent.save_weights(weights_filename)
print("Pesos guardados en:", weights_filename)

# Guardamos también el histórico de métricas, por si quieres recargarlo más tarde sin reentrenar

with open(history_filename, "w", encoding="utf-8") as f:
    json.dump(history_ddqn, f)
print("Histórico de entrenamiento guardado en:", history_filename)

```

Intervalo 1/60: 100%  10000/10000 [00:19<00:00, 507.58step/s, reward=203.1, loss=0.0000, mean_q=0.000, eps=0.985]
 Step 10,000/600,000 | reward(últ.20 ep): 203.12 | loss: 0.0000 | eps: 0.985 | tiempo intervalo: 19.9s | tiempo total: 19.9s

Intervalo 2/60: 100%  10000/10000 [00:19<00:00, 508.08step/s, reward=176.8, loss=0.0000, mean_q=0.000, eps=0.970]
 Step 20,000/600,000 | reward(últ.20 ep): 176.75 | loss: 0.0000 | eps: 0.970 | tiempo intervalo: 19.8s | tiempo total: 39.7s

Intervalo 3/60: 100%  10000/10000 [00:47<00:00, 213.80step/s, reward=127.8, loss=0.0202, mean_q=-0.013, eps=0.955]
 Step 30,000/600,000 | reward(últ.20 ep): 127.75 | loss: 0.0112 | eps: 0.955 | tiempo intervalo: 47.1s | tiempo total: 86.9s

Intervalo 4/60: 100%  10000/10000 [00:46<00:00, 219.05step/s, reward=131.0, loss=0.0001, mean_q=0.016, eps=0.940]
 Step 40,000/600,000 | reward(últ.20 ep): 131.00 | loss: 0.0094 | eps: 0.940 | tiempo intervalo: 47.0s | tiempo total: 133.8s

Intervalo 5/60: 100%  10000/10000 [00:46<00:00, 220.25step/s, reward=146.5, loss=0.0004, mean_q=0.036, eps=0.925]
 Step 50,000/600,000 | reward(últ.20 ep): 146.50 | loss: 0.0091 | eps: 0.925 | tiempo intervalo: 47.0s | tiempo total: 180.8s

Intervalo 6/60: 100%  10000/10000 [00:46<00:00, 214.02step/s, reward=154.2, loss=0.0150, mean_q=0.048, eps=0.910]
 Step 60,000/600,000 | reward(últ.20 ep): 154.25 | loss: 0.0084 | eps: 0.910 | tiempo intervalo: 46.8s | tiempo total: 227.7s

Intervalo 7/60: 100%  10000/10000 [00:47<00:00, 215.75step/s, reward=215.2, loss=0.0035, mean_q=0.122, eps=0.895]
 Step 70,000/600,000 | reward(últ.20 ep): 215.25 | loss: 0.0076 | eps: 0.895 | tiempo intervalo: 47.1s | tiempo total: 274.8s

Intervalo 8/60: 100%  10000/10000 [00:47<00:00, 213.57step/s, reward=132.2, loss=0.0018, mean_q=0.085, eps=0.880]
 Step 80,000/600,000 | reward(últ.20 ep): 132.25 | loss: 0.0066 | eps: 0.880 | tiempo intervalo: 47.1s | tiempo total: 321.9s

Intervalo 9/60: 100%  10000/10000 [00:46<00:00, 216.58step/s, reward=119.5, loss=0.0006, mean_q=0.119, eps=0.865]
Step 90,000/600,000 | reward(últ.20 ep): 119.50 | loss: 0.0069 | eps: 0.865 | tiempo intervalo: 46.9s | tiempo total: 368.8s

Intervalo 10/60: 100%  10000/10000 [00:47<00:00, 209.74step/s, reward=149.0, loss=0.0101, mean_q=0.261, eps=0.850]
Step 100,000/600,000 | reward(últ.20 ep): 149.00 | loss: 0.0072 | eps: 0.850 | tiempo intervalo: 47.4s | tiempo total: 416.2s

Intervalo 11/60: 100%  10000/10000 [00:47<00:00, 213.40step/s, reward=182.0, loss=0.0036, mean_q=0.192, eps=0.835]
Step 110,000/600,000 | reward(últ.20 ep): 182.00 | loss: 0.0076 | eps: 0.835 | tiempo intervalo: 47.2s | tiempo total: 463.4s

Intervalo 12/60: 100%  10000/10000 [00:47<00:00, 215.28step/s, reward=140.8, loss=0.0093, mean_q=0.182, eps=0.820]
Step 120,000/600,000 | reward(últ.20 ep): 140.75 | loss: 0.0077 | eps: 0.820 | tiempo intervalo: 47.5s | tiempo total: 510.9s

Intervalo 13/60: 100%  10000/10000 [00:47<00:00, 208.02step/s, reward=125.0, loss=0.0042, mean_q=0.182, eps=0.805]
Step 130,000/600,000 | reward(últ.20 ep): 125.00 | loss: 0.0075 | eps: 0.805 | tiempo intervalo: 47.5s | tiempo total: 558.4s

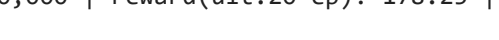
Intervalo 14/60: 100%  10000/10000 [00:47<00:00, 210.45step/s, reward=130.5, loss=0.0029, mean_q=0.197, eps=0.790]
Step 140,000/600,000 | reward(últ.20 ep): 130.50 | loss: 0.0070 | eps: 0.790 | tiempo intervalo: 47.6s | tiempo total: 605.9s

Intervalo 15/60: 100%  10000/10000 [00:47<00:00, 213.50step/s, reward=136.0, loss=0.0010, mean_q=0.204, eps=0.775]
Step 150,000/600,000 | reward(últ.20 ep): 136.00 | loss: 0.0070 | eps: 0.775 | tiempo intervalo: 47.4s | tiempo total: 653.4s

Intervalo 16/60: 100%  10000/10000 [00:47<00:00, 209.72step/s, reward=141.2, loss=0.0027, mean_q=0.362, eps=0.760]
Step 160,000/600,000 | reward(últ.20 ep): 141.25 | loss: 0.0078 | eps: 0.760 | tiempo intervalo: 47.6s | tiempo total: 701.0s

Intervalo 17/60: 100%  10000/10000 [00:47<00:00, 217.97step/s, reward=201.8, loss=0.0033, mean_q=0.274, eps=0.745]
Step 170,000/600,000 | reward(últ.20 ep): 201.75 | loss: 0.0081 | eps: 0.745 | tiempo intervalo: 47.3s | tiempo total: 748.3s

Intervalo 18/60: 100%  10000/10000 [00:47<00:00, 210.90step/s, reward=178.2, loss=0.0054, mean_q=0.340, eps=0.730]
Step 180,000/600,000 | reward(últ.20 ep): 178.25 | loss: 0.0081 | eps: 0.730 | tiempo intervalo: 47.4s | tiempo total: 795.7s

Intervalo 19/60: 100%  10000/10000 [00:47<00:00, 214.14step/s, reward=188.0, loss=0.0097, mean_q=0.202, eps=0.715]

Step 190,000/600,000 | reward(últ.20 ep): 188.00 | loss: 0.0083 | eps: 0.715 | tiempo intervalo: 47.4s | tiempo total: 84 3.1s

Intervalo 20/60: 100%  10000/10000 [00:46<00:00, 214.86step/s, reward=199.0, loss=0.0120, mean_q=0.313, eps=0.700]

Step 200,000/600,000 | reward(últ.20 ep): 199.00 | loss: 0.0087 | eps: 0.700 | tiempo intervalo: 47.0s | tiempo total: 89 0.1s

Intervalo 21/60: 100%  10000/10000 [00:48<00:00, 212.52step/s, reward=136.5, loss=0.0021, mean_q=0.348, eps=0.685]

Step 210,000/600,000 | reward(últ.20 ep): 136.50 | loss: 0.0081 | eps: 0.685 | tiempo intervalo: 48.6s | tiempo total: 93 8.7s

Intervalo 22/60: 100%  10000/10000 [00:47<00:00, 211.74step/s, reward=137.5, loss=0.0121, mean_q=0.429, eps=0.670]

Step 220,000/600,000 | reward(últ.20 ep): 137.50 | loss: 0.0081 | eps: 0.670 | tiempo intervalo: 47.7s | tiempo total: 98 6.4s

Intervalo 23/60: 100%  10000/10000 [00:48<00:00, 204.72step/s, reward=205.8, loss=0.0052, mean_q=0.490, eps=0.655]

Step 230,000/600,000 | reward(últ.20 ep): 205.75 | loss: 0.0085 | eps: 0.655 | tiempo intervalo: 48.3s | tiempo total: 103 4.7s

Intervalo 24/60: 100%  10000/10000 [00:47<00:00, 220.93step/s, reward=170.0, loss=0.0063, mean_q=0.723, eps=0.640]

Step 240,000/600,000 | reward(últ.20 ep): 170.00 | loss: 0.0091 | eps: 0.640 | tiempo intervalo: 48.0s | tiempo total: 108 2.7s

Intervalo 25/60: 100%  10000/10000 [00:47<00:00, 207.66step/s, reward=232.2, loss=0.0029, mean_q=0.600, eps=0.625]

Step 250,000/600,000 | reward(últ.20 ep): 232.25 | loss: 0.0097 | eps: 0.625 | tiempo intervalo: 47.6s | tiempo total: 113 0.3s

Intervalo 26/60: 100%  10000/10000 [00:47<00:00, 218.09step/s, reward=207.2, loss=0.0118, mean_q=0.671, eps=0.610]

Step 260,000/600,000 | reward(últ.20 ep): 207.25 | loss: 0.0095 | eps: 0.610 | tiempo intervalo: 47.4s | tiempo total: 117 7.7s

Intervalo 27/60: 100%  10000/10000 [00:47<00:00, 211.75step/s, reward=225.5, loss=0.0099, mean_q=0.733, eps=0.595]

Step 270,000/600,000 | reward(últ.20 ep): 225.50 | loss: 0.0103 | eps: 0.595 | tiempo intervalo: 47.7s | tiempo total: 122 5.3s

Intervalo 28/60: 100%  10000/10000 [00:47<00:00, 208.81step/s, reward=242.0, loss=0.0048, mean_q=0.650, eps=0.580]

Step 280,000/600,000 | reward(últ.20 ep): 242.00 | loss: 0.0101 | eps: 0.580 | tiempo intervalo: 47.5s | tiempo total: 127 2.8s

Intervalo 29/60: 100%  10000/10000 [00:48<00:00, 201.89step/s, reward=244.0, loss=0.0344, mean_q=0.774, eps=0.565]

Step 290,000/600,000 | reward(últ.20 ep): 244.00 | loss: 0.0103 | eps: 0.565 | tiempo intervalo: 48.1s | tiempo total: 132 0.9s

Intervalo 30/60: 100%  10000/10000 [00:48<00:00, 205.02step/s, reward=184.0, loss=0.0307, mean_q=0.866, eps=0.550]
Step 300,000/600,000 | reward(últ.20 ep): 184.00 | loss: 0.0107 | eps: 0.550 | tiempo intervalo: 48.4s | tiempo total: 136 9.2s

Intervalo 31/60: 100%  10000/10000 [00:50<00:00, 194.96step/s, reward=237.8, loss=0.0036, mean_q=0.698, eps=0.535]
Step 310,000/600,000 | reward(últ.20 ep): 237.75 | loss: 0.0107 | eps: 0.535 | tiempo intervalo: 50.3s | tiempo total: 141 9.6s

Intervalo 32/60: 100%  10000/10000 [00:48<00:00, 213.08step/s, reward=228.8, loss=0.0076, mean_q=0.927, eps=0.520]
Step 320,000/600,000 | reward(últ.20 ep): 228.75 | loss: 0.0111 | eps: 0.520 | tiempo intervalo: 48.7s | tiempo total: 146 8.3s

Intervalo 33/60: 100%  10000/10000 [00:47<00:00, 212.59step/s, reward=246.8, loss=0.0128, mean_q=0.984, eps=0.505]
Step 330,000/600,000 | reward(últ.20 ep): 246.75 | loss: 0.0110 | eps: 0.505 | tiempo intervalo: 48.0s | tiempo total: 151 6.3s

Intervalo 34/60: 100%  10000/10000 [00:47<00:00, 212.90step/s, reward=194.8, loss=0.0109, mean_q=1.140, eps=0.490]
Step 340,000/600,000 | reward(últ.20 ep): 194.75 | loss: 0.0104 | eps: 0.490 | tiempo intervalo: 48.0s | tiempo total: 156 4.2s

Intervalo 35/60: 100%  10000/10000 [00:47<00:00, 208.16step/s, reward=213.8, loss=0.0042, mean_q=0.912, eps=0.475]
Step 350,000/600,000 | reward(últ.20 ep): 213.75 | loss: 0.0102 | eps: 0.475 | tiempo intervalo: 47.8s | tiempo total: 161 2.0s

Intervalo 36/60: 100%  10000/10000 [00:47<00:00, 207.71step/s, reward=214.5, loss=0.0279, mean_q=1.139, eps=0.460]
Step 360,000/600,000 | reward(últ.20 ep): 214.50 | loss: 0.0101 | eps: 0.460 | tiempo intervalo: 47.8s | tiempo total: 165 9.9s

Intervalo 37/60: 100%  10000/10000 [00:47<00:00, 209.67step/s, reward=244.8, loss=0.0181, mean_q=1.189, eps=0.445]
Step 370,000/600,000 | reward(últ.20 ep): 244.75 | loss: 0.0108 | eps: 0.445 | tiempo intervalo: 47.8s | tiempo total: 170 7.7s

Intervalo 38/60: 100%  10000/10000 [00:48<00:00, 207.25step/s, reward=201.5, loss=0.0049, mean_q=1.274, eps=0.430]
Step 380,000/600,000 | reward(últ.20 ep): 201.50 | loss: 0.0109 | eps: 0.430 | tiempo intervalo: 48.1s | tiempo total: 175 5.8s

Intervalo 39/60: 100%  10000/10000 [00:47<00:00, 211.88step/s, reward=233.8, loss=0.0159, mean_q=1.430, eps=0.415]
Step 390,000/600,000 | reward(últ.20 ep): 233.75 | loss: 0.0116 | eps: 0.415 | tiempo intervalo: 47.8s | tiempo total: 180 3.5s

Intervalo 40/60: 100%  10000/10000 [00:48<00:00, 210.54step/s, reward=234.2, loss=0.0100, mean_q=1.281, eps=0.400]

Step 400,000/600,000 | reward(últ.20 ep): 234.25 | loss: 0.0120 | eps: 0.400 | tiempo intervalo: 48.3s | tiempo total: 185 1.8s

Intervalo 41/60: 100%  10000/10000 [00:48<00:00, 210.79step/s, reward=230.8, loss=0.0078, mean_q=1.091, eps=0.385]

Step 410,000/600,000 | reward(últ.20 ep): 230.75 | loss: 0.0120 | eps: 0.385 | tiempo intervalo: 48.7s | tiempo total: 190 0.5s

Intervalo 42/60: 100%  10000/10000 [00:50<00:00, 186.90step/s, reward=313.2, loss=0.0143, mean_q=1.552, eps=0.370]

Step 420,000/600,000 | reward(últ.20 ep): 313.25 | loss: 0.0120 | eps: 0.370 | tiempo intervalo: 50.1s | tiempo total: 195 0.6s

Intervalo 43/60: 100%  10000/10000 [00:50<00:00, 203.14step/s, reward=281.5, loss=0.0112, mean_q=1.619, eps=0.355]

Step 430,000/600,000 | reward(últ.20 ep): 281.50 | loss: 0.0119 | eps: 0.355 | tiempo intervalo: 50.9s | tiempo total: 200 1.4s

Intervalo 44/60: 100%  10000/10000 [00:49<00:00, 205.75step/s, reward=233.8, loss=0.0320, mean_q=1.295, eps=0.340]

Step 440,000/600,000 | reward(últ.20 ep): 233.75 | loss: 0.0118 | eps: 0.340 | tiempo intervalo: 49.2s | tiempo total: 205 0.6s

Intervalo 45/60: 100%  10000/10000 [00:48<00:00, 211.58step/s, reward=296.2, loss=0.0307, mean_q=1.325, eps=0.325]

Step 450,000/600,000 | reward(últ.20 ep): 296.25 | loss: 0.0119 | eps: 0.325 | tiempo intervalo: 48.6s | tiempo total: 209 9.2s

Intervalo 46/60: 100%  10000/10000 [00:48<00:00, 208.71step/s, reward=268.8, loss=0.0092, mean_q=1.152, eps=0.310]

Step 460,000/600,000 | reward(últ.20 ep): 268.75 | loss: 0.0128 | eps: 0.310 | tiempo intervalo: 48.4s | tiempo total: 214 7.6s

Intervalo 47/60: 100%  10000/10000 [00:48<00:00, 210.27step/s, reward=342.2, loss=0.0245, mean_q=1.568, eps=0.295]

Step 470,000/600,000 | reward(últ.20 ep): 342.25 | loss: 0.0139 | eps: 0.295 | tiempo intervalo: 48.1s | tiempo total: 219 5.6s

Intervalo 48/60: 100%  10000/10000 [00:48<00:00, 212.34step/s, reward=322.0, loss=0.0379, mean_q=1.751, eps=0.280]

Step 480,000/600,000 | reward(últ.20 ep): 322.00 | loss: 0.0144 | eps: 0.280 | tiempo intervalo: 48.1s | tiempo total: 224 3.8s

Intervalo 49/60: 100%  10000/10000 [00:48<00:00, 206.37step/s, reward=290.2, loss=0.0053, mean_q=1.635, eps=0.265]

Step 490,000/600,000 | reward(últ.20 ep): 290.25 | loss: 0.0142 | eps: 0.265 | tiempo intervalo: 48.4s | tiempo total: 229 2.1s

Intervalo 50/60: 100%  10000/10000 [00:48<00:00, 208.36step/s, reward=335.0, loss=0.0117, mean_q=1.377, eps=0.250]

Step 500,000/600,000 | reward(últ.20 ep): 335.00 | loss: 0.0145 | eps: 0.250 | tiempo intervalo: 48.5s | tiempo total: 234 0.6s

Intervalo 51/60: 100%  10000/10000 [00:48<00:00, 212.77step/s, reward=298.8, loss=0.0228, mean_q=1.694, eps=0.235]
Step 510,000/600,000 | reward(últ.20 ep): 298.75 | loss: 0.0144 | eps: 0.235 | tiempo intervalo: 48.6s | tiempo total: 238 9.2s

Intervalo 52/60: 100%  10000/10000 [00:50<00:00, 203.69step/s, reward=322.2, loss=0.0097, mean_q=1.820, eps=0.220]
Step 520,000/600,000 | reward(últ.20 ep): 322.25 | loss: 0.0150 | eps: 0.220 | tiempo intervalo: 50.0s | tiempo total: 243 9.3s

Intervalo 53/60: 100%  10000/10000 [00:48<00:00, 208.50step/s, reward=382.2, loss=0.0091, mean_q=1.454, eps=0.205]
Step 530,000/600,000 | reward(últ.20 ep): 382.25 | loss: 0.0158 | eps: 0.205 | tiempo intervalo: 48.9s | tiempo total: 248 8.1s

Intervalo 54/60: 100%  10000/10000 [00:48<00:00, 206.67step/s, reward=293.2, loss=0.0141, mean_q=1.737, eps=0.190]
Step 540,000/600,000 | reward(últ.20 ep): 293.25 | loss: 0.0154 | eps: 0.190 | tiempo intervalo: 48.7s | tiempo total: 253 6.8s

Intervalo 55/60: 100%  10000/10000 [00:48<00:00, 203.46step/s, reward=366.0, loss=0.0172, mean_q=1.735, eps=0.175]
Step 550,000/600,000 | reward(últ.20 ep): 366.00 | loss: 0.0158 | eps: 0.175 | tiempo intervalo: 48.7s | tiempo total: 258 5.5s

Intervalo 56/60: 100%  10000/10000 [00:48<00:00, 205.81step/s, reward=413.0, loss=0.0055, mean_q=1.861, eps=0.160]
Step 560,000/600,000 | reward(últ.20 ep): 413.00 | loss: 0.0163 | eps: 0.160 | tiempo intervalo: 48.7s | tiempo total: 263 4.2s

Intervalo 57/60: 100%  10000/10000 [00:48<00:00, 209.80step/s, reward=339.8, loss=0.0045, mean_q=1.626, eps=0.145]
Step 570,000/600,000 | reward(últ.20 ep): 339.75 | loss: 0.0158 | eps: 0.145 | tiempo intervalo: 49.0s | tiempo total: 268 3.2s

Intervalo 58/60: 100%  10000/10000 [00:48<00:00, 201.35step/s, reward=318.0, loss=0.0058, mean_q=1.430, eps=0.130]
Step 580,000/600,000 | reward(últ.20 ep): 318.00 | loss: 0.0161 | eps: 0.130 | tiempo intervalo: 48.9s | tiempo total: 273 2.1s

Intervalo 59/60: 100%  10000/10000 [00:48<00:00, 203.28step/s, reward=392.0, loss=0.0056, mean_q=1.718, eps=0.115]
Step 590,000/600,000 | reward(últ.20 ep): 392.00 | loss: 0.0173 | eps: 0.115 | tiempo intervalo: 48.8s | tiempo total: 278 0.9s

Intervalo 60/60: 100%  10000/10000 [00:48<00:00, 208.76step/s, reward=382.2, loss=0.0137, mean_q=1.860, eps=0.100]

Step 600,000/600,000 | reward(últ.20 ep): 382.25 | loss: 0.0164 | eps: 0.100 | tiempo intervalo: 48.7s | tiempo total: 2829.6s

Entrenamiento finalizado. Tiempo total: 2829.6s (47.16 min, 0.79 h)

Pesos guardados en: ddqn_ALE_SpaceInvaders-v5_weights.pt

Histórico de entrenamiento guardado en: history_ddqn_SpaceInvaders.json

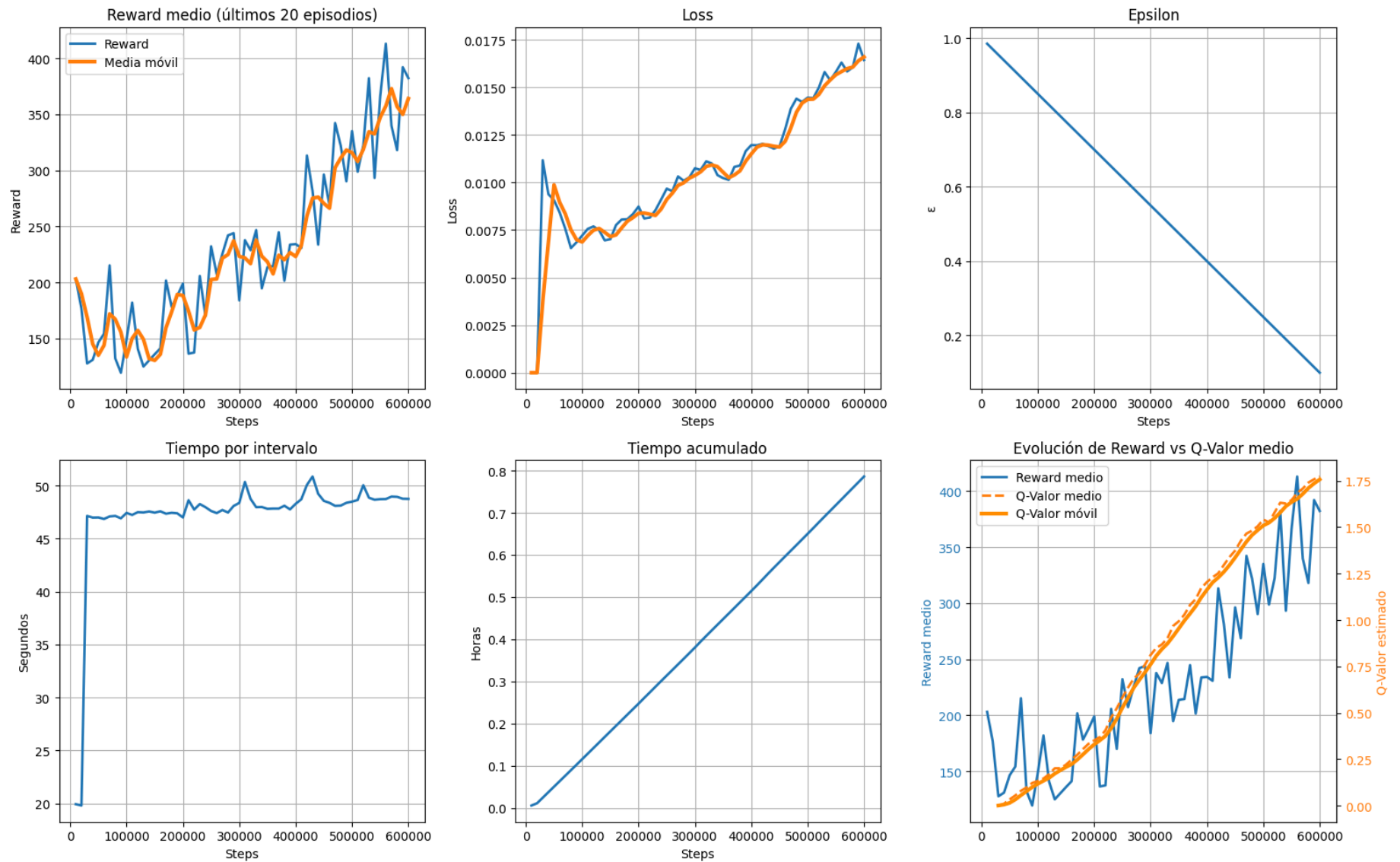
Resultados

```
In [ ]: # Siempre traemos los datos del .json

history_filename = "history_ddqn_SpaceInvaders.json"

with open(history_filename, "r", encoding="utf-8") as f:
    history = json.load(f)

plot_training_history(history)
```



3.3.2 - Dueling DQN

En Space Invaders, hay muchos estados en los que las acciones no tienen una importancia significativa. Por ejemplo, cuando no existen enemigos cerca moverse a izquierda o a derecha puede producir el mismo resultado. En DQN, la red intenta aprender el valor de cada acción

de manera independiente, incluso cuando apenas existen diferencias entre ellas. Esto supone un esfuerzo de aprendizaje innecesario.

Para sacar partida de esta casuística y mejorar el desempeño en el aprendizaje, se propone utilizar Dueling DQN, que sustituye la estimación del valor Q de cada acción por la estimación del valor del estado y de la ventaja de cada acción. De esta manera:

- La estimación del valor del estado determina lo beneficiosa que es la situación independiente de la acción tomada.
- La estimación de la ventaja indica cuánto mejora o empeora una acción concreta respecto a las demás.

Definición

```
In [ ]: class DuelingDQNCNN(nn.Module):
    """Red con arquitectura Dueling: separa la estimación en V(s) y A(s,a).
    Q(s,a) = V(s) + (A(s,a) - media_a A(s,a))
    La resta de la media evita el problema de identificabilidad (que V y A
    absorban valores arbitrarios que se compensen entre sí)."""

    def __init__(self, n_actions, window_length=WINDOW_LENGTH):
        super().__init__()
        self.conv1 = nn.Conv2d(window_length, 32, kernel_size=8, stride=4)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1)

        conv_out_size = self._get_conv_out((window_length, 84, 84))

        # Rama de Valor: V(s) -> un único escalar
        self.value_fc1 = nn.Linear(conv_out_size, 512)
        self.value_fc2 = nn.Linear(512, 1)

        # Rama de Ventaja: A(s,a) -> un valor por acción
        self.adv_fc1 = nn.Linear(conv_out_size, 512)
        self.adv_fc2 = nn.Linear(512, n_actions)

    def _get_conv_out(self, shape):
        o = torch.zeros(1, *shape)
        o = self.conv3(self.conv2(self.conv1(o)))
        return int(np.prod(o.shape))

    def forward(self, x):
```

```

x = F.relu(self.conv1(x))
x = F.relu(self.conv2(x))
x = F.relu(self.conv3(x))
x = x.view(x.size(0), -1)

v = F.relu(self.value_fc1(x))
v = self.value_fc2(v)                # (batch, 1)

a = F.relu(self.adv_fc1(x))
a = self.adv_fc2(a)                  # (batch, n_actions)

q = v + (a - a.mean(dim=1, keepdim=True))
return q

```

```

model_dueling = DuelingDQNCNN(N_ACTIONS, WINDOW_LENGTH).to(device)
print(model_dueling)
n_params = sum(p.numel() for p in model_dueling.parameters())
print(f"Total de parámetros: {n_params:,}")

```

```

DuelingDQNCNN(
  (conv1): Conv2d(4, 32, kernel_size=(8, 8), stride=(4, 4))
  (conv2): Conv2d(32, 64, kernel_size=(4, 4), stride=(2, 2))
  (conv3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
  (value_fc1): Linear(in_features=3136, out_features=512, bias=True)
  (value_fc2): Linear(in_features=512, out_features=1, bias=True)
  (adv_fc1): Linear(in_features=3136, out_features=512, bias=True)
  (adv_fc2): Linear(in_features=512, out_features=6, bias=True)
)
Total de parámetros: 3,293,863

```

```

In [ ]: class DuelingDQNAgent(DQNAgent):
        """
        Agente DQN que usa la arquitectura Dueling (DuelingDQNCNN) tanto en la
        red principal como en la red objetivo. La regla de entrenamiento
        (train_step) es la del DQN estándar, heredada sin cambios.
        """

        def __init__(self, model, n_actions, memory, policy, device,
                      gamma=0.99, lr=2.5e-4, nb_steps_warmup=20_000,
                      target_model_update=10_000, train_interval=4, batch_size=32):

```

```

self.device = device
self.model = model.to(device)
self.target_model = DuelingDQNCNN(n_actions, WINDOW_LENGTH).to(device)
self.target_model.load_state_dict(self.model.state_dict())
self.target_model.eval()

self.n_actions = n_actions
self.memory = memory
self.policy = policy
self.gamma = gamma
self.nb_steps_warmup = nb_steps_warmup
self.target_model_update = target_model_update
self.train_interval = train_interval
self.batch_size = batch_size

self.optimizer = optim.Adam(self.model.parameters(), lr=lr)
self.loss_fn = nn.SmoothL1Loss()

self.step_count = 0

```

```

In [ ]: NB_STEPS = 600000

weights_filename = f"dueling_{ENV_NAME.replace('/', '_')}_weights.pt"
history_filename = "history_dueling_SpaceInvaders.json"

print(f"Pasos de entrenamiento: {NB_STEPS:,}")

```

Pasos de entrenamiento: 600,000

Instanciación y entreno

```

In [ ]: processor = AtariProcessor(INPUT_SHAPE, WINDOW_LENGTH)

memory = ReplayMemory(capacity=1_000_000,
                      state_shape=(WINDOW_LENGTH, 84, 84))

policy = LinearAnnealedEpsGreedy(
    value_max=1.0,
    value_min=0.1,
    value_test=0.05,

```



```

        nb_steps=NB_STEPS,
    )

    agent = DuelingDQNAgent(
        model=model_dueling,
        n_actions=N_ACTIONS,
        memory=memory,
        policy=policy,
        device=device,
        gamma=0.99,
        lr=2.5e-4,
        nb_steps_warmup=NB_STEPS_WARMUP,
        target_model_update=10_000,
        train_interval=4,
        batch_size=32,
    )

    agent, history_dueling = train(
        env,
        agent,
        processor,
        nb_steps=NB_STEPS,
    )

    agent.save_weights(weights_filename)
    print("Pesos guardados en:", weights_filename)

    with open(history_filename, "w", encoding="utf-8") as f:
        json.dump(history_dueling, f)
    print("Histórico de entrenamiento guardado en:", history_filename)

```

Intervalo 1/60: 100%  10000/10000 [00:19<00:00, 475.31step/s, reward=126.8, loss=0.0000, mean_q=0.000, eps=0.985]

Step 10,000/600,000 | reward(últ.20 ep): 126.84 | loss: 0.0000 | eps: 0.985 | tiempo intervalo: 20.0s | tiempo total: 20.0s

Intervalo 2/60: 100%  10000/10000 [00:19<00:00, 517.73step/s, reward=174.8, loss=0.0000, mean_q=0.000, eps=0.970]

Step 20,000/600,000 | reward(últ.20 ep): 174.75 | loss: 0.0000 | eps: 0.970 | tiempo intervalo: 19.8s | tiempo total: 39.8s

Intervalo 3/60: 100%  10000/10000 [00:48<00:00, 202.25step/s, reward=175.8, loss=0.0003, mean_q=0.039, eps=0.955]

Step 30,000/600,000 | reward(últ.20 ep): 175.75 | loss: 0.0088 | eps: 0.955 | tiempo intervalo: 48.9s | tiempo total: 8 8.6s

Intervalo 4/60: 100%  10000/10000 [00:48<00:00, 207.56step/s, reward=172.8, loss=0.0081, mean_q=0.062, eps=0.940]

Step 40,000/600,000 | reward(últ.20 ep): 172.75 | loss: 0.0085 | eps: 0.940 | tiempo intervalo: 49.0s | tiempo total: 13 7.6s

Intervalo 5/60: 100%  10000/10000 [00:49<00:00, 207.01step/s, reward=134.8, loss=0.0313, mean_q=0.044, eps=0.925]

Step 50,000/600,000 | reward(últ.20 ep): 134.75 | loss: 0.0067 | eps: 0.925 | tiempo intervalo: 49.2s | tiempo total: 18 6.8s

Intervalo 6/60: 100%  10000/10000 [00:48<00:00, 205.56step/s, reward=151.2, loss=0.0034, mean_q=0.066, eps=0.910]

Step 60,000/600,000 | reward(últ.20 ep): 151.25 | loss: 0.0069 | eps: 0.910 | tiempo intervalo: 48.8s | tiempo total: 23 5.6s

Intervalo 7/60: 100%  10000/10000 [00:49<00:00, 209.74step/s, reward=128.2, loss=0.0055, mean_q=0.087, eps=0.895]

Step 70,000/600,000 | reward(últ.20 ep): 128.25 | loss: 0.0063 | eps: 0.895 | tiempo intervalo: 49.2s | tiempo total: 28 4.8s

Intervalo 8/60: 100%  10000/10000 [00:49<00:00, 191.90step/s, reward=126.5, loss=0.0009, mean_q=0.098, eps=0.880]

Step 80,000/600,000 | reward(últ.20 ep): 126.50 | loss: 0.0056 | eps: 0.880 | tiempo intervalo: 49.6s | tiempo total: 33 4.4s

Intervalo 9/60: 100%  10000/10000 [00:49<00:00, 211.29step/s, reward=132.5, loss=0.0021, mean_q=0.100, eps=0.865]

Step 90,000/600,000 | reward(últ.20 ep): 132.50 | loss: 0.0058 | eps: 0.865 | tiempo intervalo: 49.7s | tiempo total: 38 4.1s

Intervalo 10/60: 100%  10000/10000 [00:49<00:00, 205.76step/s, reward=185.2, loss=0.0213, mean_q=0.181, eps=0.850]

Step 100,000/600,000 | reward(últ.20 ep): 185.25 | loss: 0.0058 | eps: 0.850 | tiempo intervalo: 49.0s | tiempo total: 43 3.1s

Intervalo 11/60: 100%  10000/10000 [00:48<00:00, 205.23step/s, reward=136.2, loss=0.0008, mean_q=0.129, eps=0.835]

Step 110,000/600,000 | reward(últ.20 ep): 136.25 | loss: 0.0066 | eps: 0.835 | tiempo intervalo: 48.8s | tiempo total: 48 2.0s

Intervalo 12/60: 100%  10000/10000 [00:48<00:00, 205.40step/s, reward=168.5, loss=0.0041, mean_q=0.150, eps=0.820]

Step 120,000/600,000 | reward(últ.20 ep): 168.50 | loss: 0.0069 | eps: 0.820 | tiempo intervalo: 48.5s | tiempo total: 53 0.5s

Intervalo 13/60: 100%  10000/10000 [00:49<00:00, 209.40step/s, reward=158.2, loss=0.0013, mean_q=0.130, eps=0.805]

Step 130,000/600,000 | reward(últ.20 ep): 158.25 | loss: 0.0070 | eps: 0.805 | tiempo intervalo: 49.6s | tiempo total: 58 0.1s

Intervalo 14/60: 100%  10000/10000 [00:48<00:00, 214.52step/s, reward=159.0, loss=0.0015, mean_q=0.280, eps=0.790]
Step 140,000/600,000 | reward(últ.20 ep): 159.00 | loss: 0.0069 | eps: 0.790 | tiempo intervalo: 48.7s | tiempo total: 628.8s

Intervalo 15/60: 100%  10000/10000 [00:50<00:00, 200.85step/s, reward=133.2, loss=0.0059, mean_q=0.345, eps=0.775]
Step 150,000/600,000 | reward(últ.20 ep): 133.25 | loss: 0.0067 | eps: 0.775 | tiempo intervalo: 50.1s | tiempo total: 678.9s

Intervalo 16/60: 100%  10000/10000 [00:50<00:00, 201.56step/s, reward=159.0, loss=0.0061, mean_q=0.327, eps=0.760]
Step 160,000/600,000 | reward(últ.20 ep): 159.00 | loss: 0.0076 | eps: 0.760 | tiempo intervalo: 50.1s | tiempo total: 729.0s

Intervalo 17/60: 100%  10000/10000 [00:49<00:00, 199.63step/s, reward=190.0, loss=0.0077, mean_q=0.271, eps=0.745]
Step 170,000/600,000 | reward(últ.20 ep): 190.00 | loss: 0.0078 | eps: 0.745 | tiempo intervalo: 49.8s | tiempo total: 778.8s

Intervalo 18/60: 100%  10000/10000 [00:49<00:00, 204.94step/s, reward=186.2, loss=0.0018, mean_q=0.316, eps=0.730]
Step 180,000/600,000 | reward(últ.20 ep): 186.25 | loss: 0.0074 | eps: 0.730 | tiempo intervalo: 49.8s | tiempo total: 828.6s

Intervalo 19/60: 100%  10000/10000 [00:49<00:00, 207.78step/s, reward=159.5, loss=0.0224, mean_q=0.372, eps=0.715]
Step 190,000/600,000 | reward(últ.20 ep): 159.50 | loss: 0.0077 | eps: 0.715 | tiempo intervalo: 49.5s | tiempo total: 878.2s

Intervalo 20/60: 100%  10000/10000 [00:49<00:00, 200.06step/s, reward=158.0, loss=0.0035, mean_q=0.407, eps=0.700]
Step 200,000/600,000 | reward(últ.20 ep): 158.00 | loss: 0.0087 | eps: 0.700 | tiempo intervalo: 49.5s | tiempo total: 927.7s

Intervalo 21/60: 100%  10000/10000 [00:50<00:00, 198.61step/s, reward=174.2, loss=0.0071, mean_q=0.456, eps=0.685]
Step 210,000/600,000 | reward(últ.20 ep): 174.25 | loss: 0.0091 | eps: 0.685 | tiempo intervalo: 50.1s | tiempo total: 977.9s

Intervalo 22/60: 100%  10000/10000 [00:51<00:00, 188.75step/s, reward=220.8, loss=0.0084, mean_q=0.584, eps=0.670]
Step 220,000/600,000 | reward(últ.20 ep): 220.75 | loss: 0.0091 | eps: 0.670 | tiempo intervalo: 51.0s | tiempo total: 1028.9s

Intervalo 23/60: 100%  10000/10000 [00:51<00:00, 198.65step/s, reward=192.8, loss=0.0052, mean_q=0.591, eps=0.655]
Step 230,000/600,000 | reward(últ.20 ep): 192.75 | loss: 0.0100 | eps: 0.655 | tiempo intervalo: 51.5s | tiempo total: 1080.3s

Intervalo 24/60: 100%  10000/10000 [00:49<00:00, 199.99step/s, reward=232.0, loss=0.0078, mean_q=0.807, eps=0.640]

Step 240,000/600,000 | reward(últ.20 ep): 232.00 | loss: 0.0095 | eps: 0.640 | tiempo intervalo: 49.8s | tiempo total: 113 0.1s

Intervalo 25/60: 100%  10000/10000 [00:49<00:00, 201.10step/s, reward=211.2, loss=0.0125, mean_q=0.693, eps=0.625]

Step 250,000/600,000 | reward(últ.20 ep): 211.25 | loss: 0.0101 | eps: 0.625 | tiempo intervalo: 49.7s | tiempo total: 117 9.8s

Intervalo 26/60: 100%  10000/10000 [00:49<00:00, 208.08step/s, reward=270.0, loss=0.0188, mean_q=0.697, eps=0.610]

Step 260,000/600,000 | reward(últ.20 ep): 270.00 | loss: 0.0104 | eps: 0.610 | tiempo intervalo: 49.5s | tiempo total: 122 9.3s

Intervalo 27/60: 100%  10000/10000 [00:49<00:00, 196.36step/s, reward=256.5, loss=0.0070, mean_q=0.839, eps=0.595]

Step 270,000/600,000 | reward(últ.20 ep): 256.50 | loss: 0.0103 | eps: 0.595 | tiempo intervalo: 49.5s | tiempo total: 127 8.8s

Intervalo 28/60: 100%  10000/10000 [00:49<00:00, 205.93step/s, reward=192.8, loss=0.0095, mean_q=0.933, eps=0.580]

Step 280,000/600,000 | reward(últ.20 ep): 192.75 | loss: 0.0103 | eps: 0.580 | tiempo intervalo: 49.7s | tiempo total: 132 8.5s

Intervalo 29/60: 100%  10000/10000 [00:49<00:00, 200.31step/s, reward=245.2, loss=0.0373, mean_q=0.855, eps=0.565]

Step 290,000/600,000 | reward(últ.20 ep): 245.25 | loss: 0.0108 | eps: 0.565 | tiempo intervalo: 49.5s | tiempo total: 137 8.0s

Intervalo 30/60: 100%  10000/10000 [00:49<00:00, 194.15step/s, reward=207.5, loss=0.0040, mean_q=0.824, eps=0.550]

Step 300,000/600,000 | reward(últ.20 ep): 207.50 | loss: 0.0110 | eps: 0.550 | tiempo intervalo: 49.8s | tiempo total: 142 7.8s

Intervalo 31/60: 100%  10000/10000 [00:50<00:00, 202.27step/s, reward=270.0, loss=0.0073, mean_q=0.753, eps=0.535]

Step 310,000/600,000 | reward(últ.20 ep): 270.00 | loss: 0.0110 | eps: 0.535 | tiempo intervalo: 50.2s | tiempo total: 147 8.0s

Intervalo 32/60: 100%  10000/10000 [00:50<00:00, 197.01step/s, reward=224.0, loss=0.0250, mean_q=1.237, eps=0.520]

Step 320,000/600,000 | reward(últ.20 ep): 224.00 | loss: 0.0113 | eps: 0.520 | tiempo intervalo: 50.3s | tiempo total: 152 8.3s

Intervalo 33/60: 100%  10000/10000 [00:52<00:00, 183.31step/s, reward=254.8, loss=0.0053, mean_q=1.010, eps=0.505]

Step 330,000/600,000 | reward(últ.20 ep): 254.75 | loss: 0.0111 | eps: 0.505 | tiempo intervalo: 53.0s | tiempo total: 158 1.3s

Intervalo 34/60: 100%  10000/10000 [00:50<00:00, 195.17step/s, reward=280.2, loss=0.0075, mean_q=1.090, eps=0.490]

Step 340,000/600,000 | reward(últ.20 ep): 280.25 | loss: 0.0117 | eps: 0.490 | tiempo intervalo: 50.6s | tiempo total: 163 1.9s

Intervalo 35/60: 100%  10000/10000 [00:49<00:00, 202.80step/s, reward=261.8, loss=0.0111, mean_q=1.147, eps=0.475]
Step 350,000/600,000 | reward(últ.20 ep): 261.75 | loss: 0.0117 | eps: 0.475 | tiempo intervalo: 49.8s | tiempo total: 168 1.7s

Intervalo 36/60: 100%  10000/10000 [00:49<00:00, 200.27step/s, reward=272.2, loss=0.0081, mean_q=1.189, eps=0.460]
Step 360,000/600,000 | reward(últ.20 ep): 272.25 | loss: 0.0117 | eps: 0.460 | tiempo intervalo: 49.9s | tiempo total: 173 1.5s

Intervalo 37/60: 100%  10000/10000 [00:49<00:00, 198.49step/s, reward=257.0, loss=0.0052, mean_q=1.386, eps=0.445]
Step 370,000/600,000 | reward(últ.20 ep): 257.00 | loss: 0.0126 | eps: 0.445 | tiempo intervalo: 49.9s | tiempo total: 178 1.4s

Intervalo 38/60: 100%  10000/10000 [00:50<00:00, 196.65step/s, reward=248.0, loss=0.0062, mean_q=1.317, eps=0.430]
Step 380,000/600,000 | reward(últ.20 ep): 248.00 | loss: 0.0125 | eps: 0.430 | tiempo intervalo: 50.1s | tiempo total: 183 1.6s

Intervalo 39/60: 100%  10000/10000 [00:49<00:00, 196.15step/s, reward=299.0, loss=0.0115, mean_q=1.084, eps=0.415]
Step 390,000/600,000 | reward(últ.20 ep): 299.00 | loss: 0.0118 | eps: 0.415 | tiempo intervalo: 50.0s | tiempo total: 188 1.5s

Intervalo 40/60: 100%  10000/10000 [00:50<00:00, 202.96step/s, reward=271.5, loss=0.0114, mean_q=1.018, eps=0.400]
Step 400,000/600,000 | reward(últ.20 ep): 271.50 | loss: 0.0117 | eps: 0.400 | tiempo intervalo: 50.1s | tiempo total: 193 1.7s

Intervalo 41/60: 100%  10000/10000 [00:50<00:00, 195.90step/s, reward=248.0, loss=0.0088, mean_q=1.586, eps=0.385]
Step 410,000/600,000 | reward(últ.20 ep): 248.00 | loss: 0.0128 | eps: 0.385 | tiempo intervalo: 50.2s | tiempo total: 198 1.9s

Intervalo 42/60: 100%  10000/10000 [00:51<00:00, 203.46step/s, reward=277.0, loss=0.0089, mean_q=1.446, eps=0.370]
Step 420,000/600,000 | reward(últ.20 ep): 277.00 | loss: 0.0128 | eps: 0.370 | tiempo intervalo: 51.5s | tiempo total: 203 3.4s

Intervalo 43/60: 100%  10000/10000 [00:50<00:00, 194.27step/s, reward=303.2, loss=0.0093, mean_q=1.617, eps=0.355]
Step 430,000/600,000 | reward(últ.20 ep): 303.25 | loss: 0.0131 | eps: 0.355 | tiempo intervalo: 50.2s | tiempo total: 208 3.6s

Intervalo 44/60: 100%  10000/10000 [00:50<00:00, 192.70step/s, reward=358.0, loss=0.0032, mean_q=1.297, eps=0.340]
Step 440,000/600,000 | reward(últ.20 ep): 358.00 | loss: 0.0135 | eps: 0.340 | tiempo intervalo: 50.9s | tiempo total: 213 4.5s

Intervalo 45/60: 100%  10000/10000 [00:51<00:00, 199.72step/s, reward=357.2, loss=0.0136, mean_q=1.407, eps=0.325]

Step 450,000/600,000 | reward(últ.20 ep): 357.25 | loss: 0.0139 | eps: 0.325 | tiempo intervalo: 51.2s | tiempo total: 218 5.7s

Intervalo 46/60: 100%  10000/10000 [00:50<00:00, 191.61step/s, reward=373.8, loss=0.0057, mean_q=1.322, eps=0.310]

Step 460,000/600,000 | reward(últ.20 ep): 373.75 | loss: 0.0143 | eps: 0.310 | tiempo intervalo: 50.9s | tiempo total: 223 6.7s

Intervalo 47/60: 100%  10000/10000 [00:50<00:00, 200.22step/s, reward=383.0, loss=0.0190, mean_q=1.555, eps=0.295]

Step 470,000/600,000 | reward(últ.20 ep): 383.00 | loss: 0.0152 | eps: 0.295 | tiempo intervalo: 50.4s | tiempo total: 228 7.1s

Intervalo 48/60: 100%  10000/10000 [00:50<00:00, 200.42step/s, reward=350.8, loss=0.0169, mean_q=1.686, eps=0.280]

Step 480,000/600,000 | reward(últ.20 ep): 350.75 | loss: 0.0154 | eps: 0.280 | tiempo intervalo: 50.7s | tiempo total: 233 7.7s

Intervalo 49/60: 100%  10000/10000 [00:50<00:00, 196.45step/s, reward=395.0, loss=0.0139, mean_q=2.389, eps=0.265]

Step 490,000/600,000 | reward(últ.20 ep): 395.00 | loss: 0.0160 | eps: 0.265 | tiempo intervalo: 50.3s | tiempo total: 238 8.1s

Intervalo 50/60: 100%  10000/10000 [00:50<00:00, 185.39step/s, reward=418.5, loss=0.0156, mean_q=1.769, eps=0.250]

Step 500,000/600,000 | reward(últ.20 ep): 418.50 | loss: 0.0168 | eps: 0.250 | tiempo intervalo: 50.6s | tiempo total: 243 8.6s

Intervalo 51/60: 100%  10000/10000 [00:50<00:00, 201.38step/s, reward=364.5, loss=0.0097, mean_q=1.824, eps=0.235]

Step 510,000/600,000 | reward(últ.20 ep): 364.50 | loss: 0.0163 | eps: 0.235 | tiempo intervalo: 50.7s | tiempo total: 248 9.3s

Intervalo 52/60: 100%  10000/10000 [00:50<00:00, 203.38step/s, reward=399.0, loss=0.0304, mean_q=1.932, eps=0.220]

Step 520,000/600,000 | reward(últ.20 ep): 399.00 | loss: 0.0167 | eps: 0.220 | tiempo intervalo: 50.7s | tiempo total: 254 0.0s

Intervalo 53/60: 100%  10000/10000 [00:50<00:00, 195.69step/s, reward=423.5, loss=0.0142, mean_q=2.087, eps=0.205]

Step 530,000/600,000 | reward(últ.20 ep): 423.50 | loss: 0.0179 | eps: 0.205 | tiempo intervalo: 50.6s | tiempo total: 259 0.6s

Intervalo 54/60: 100%  10000/10000 [00:50<00:00, 197.63step/s, reward=449.0, loss=0.0295, mean_q=1.984, eps=0.190]

Step 540,000/600,000 | reward(últ.20 ep): 449.00 | loss: 0.0174 | eps: 0.190 | tiempo intervalo: 50.6s | tiempo total: 264 1.2s

Intervalo 55/60: 100%  10000/10000 [00:51<00:00, 194.45step/s, reward=399.0, loss=0.0217, mean_q=2.076, eps=0.175]

Step 550,000/600,000 | reward(últ.20 ep): 399.00 | loss: 0.0178 | eps: 0.175 | tiempo intervalo: 51.2s | tiempo total: 269 2.5s

Intervalo 56/60: 100%  10000/10000 [00:51<00:00, 195.14step/s, reward=422.8, loss=0.0071, mean_q=1.942, eps=0.160]
 Step 560,000/600,000 | reward(últ.20 ep): 422.75 | loss: 0.0190 | eps: 0.160 | tiempo intervalo: 51.8s | tiempo total: 274 4.3s

Intervalo 57/60: 100%  10000/10000 [00:51<00:00, 183.37step/s, reward=468.0, loss=0.0136, mean_q=2.507, eps=0.145]
 Step 570,000/600,000 | reward(últ.20 ep): 468.00 | loss: 0.0188 | eps: 0.145 | tiempo intervalo: 51.9s | tiempo total: 279 6.1s

Intervalo 58/60: 100%  10000/10000 [00:51<00:00, 195.84step/s, reward=476.2, loss=0.0120, mean_q=2.431, eps=0.130]
 Step 580,000/600,000 | reward(últ.20 ep): 476.25 | loss: 0.0199 | eps: 0.130 | tiempo intervalo: 51.4s | tiempo total: 284 7.5s

Intervalo 59/60: 100%  10000/10000 [00:50<00:00, 203.02step/s, reward=399.2, loss=0.0114, mean_q=2.463, eps=0.115]
 Step 590,000/600,000 | reward(últ.20 ep): 399.25 | loss: 0.0207 | eps: 0.115 | tiempo intervalo: 50.7s | tiempo total: 289 8.2s

Intervalo 60/60: 100%  10000/10000 [00:51<00:00, 193.44step/s, reward=393.8, loss=0.0338, mean_q=2.231, eps=0.100]
 Step 600,000/600,000 | reward(últ.20 ep): 393.75 | loss: 0.0201 | eps: 0.100 | tiempo intervalo: 51.1s | tiempo total: 294 9.3s

Entrenamiento finalizado. Tiempo total: 2949.3s (49.16 min, 0.82 h)

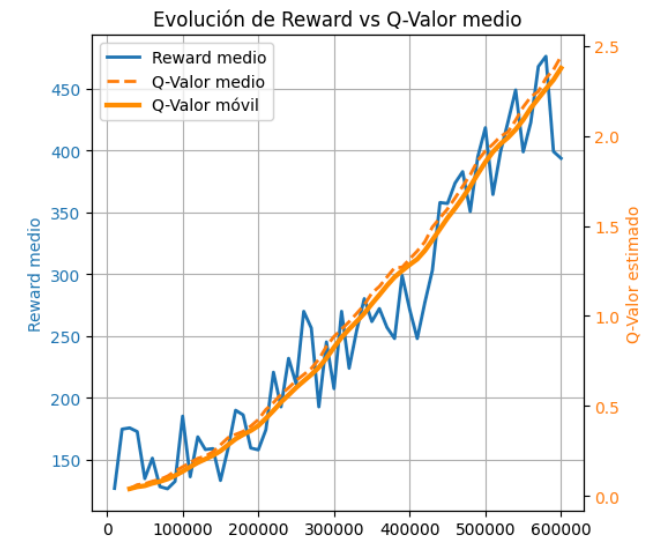
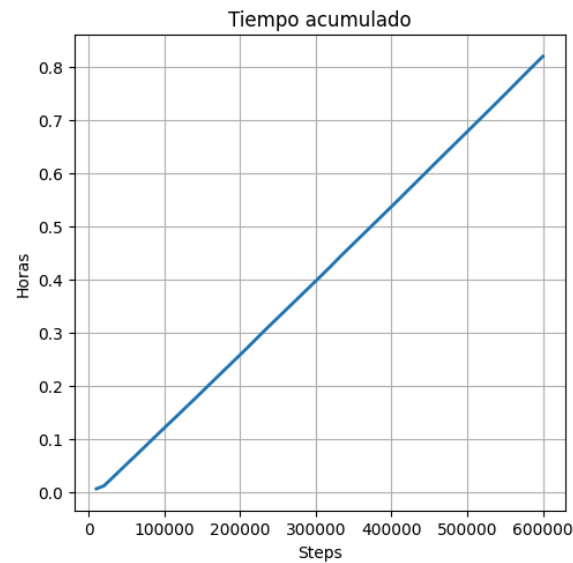
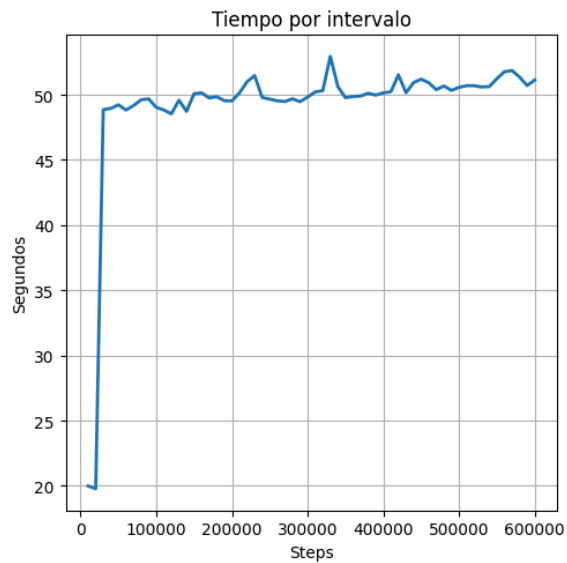
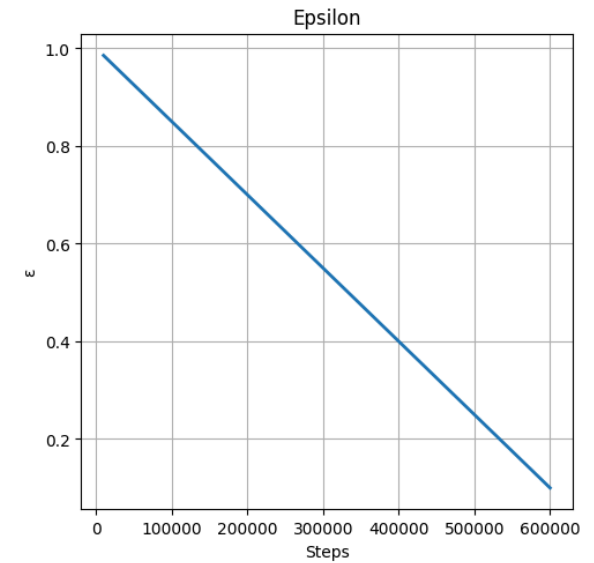
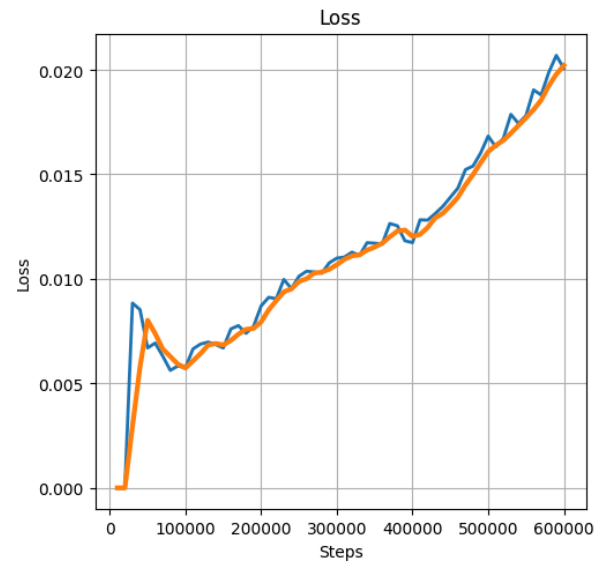
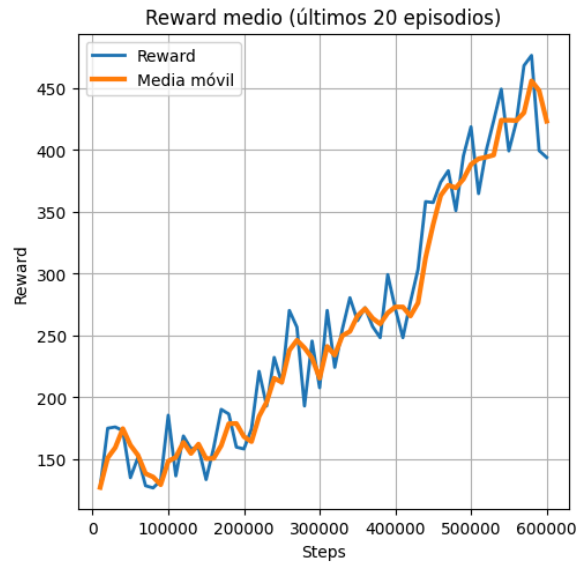
Pesos guardados en: dueling_ALE_SpaceInvaders-v5_weights.pt

Histórico de entrenamiento guardado en: history_dueling_SpaceInvaders.json

Resultados

```
In [ ]: with open("history_dueling_SpaceInvaders.json", "r", encoding="utf-8") as f:
        history = json.load(f)

        plot_training_history(history)
```



3.3.3 - Ajuste de Hiperparámetros y Funciones de Recompensa

El rendimiento de DQN depende en gran medida de los hiperparámetros utilizados durante el entrenamiento y de la función de recompensa. Ajustar el algoritmo para el caso específico de Space Invaders puede mejorar notablemente el rendimiento.

Para el ajuste de hiperparámetros se proponen los siguientes cambios con respecto a DQN:

- Reducir el Learning Rate para estabilizar más el aprendizaje y evitar sobreajustes.
- Actualizar el Target más frecuentemente para acelerar el aprendizaje.
- Aumentar el Batch Size para que el gradiente sea menos ruidos y más preciso.

Rediseño de la recompensa: En este caso, se propone penalizar la pérdida de vidas y recortar de forma estándar la recompensa en el intervalo $[-1, 1]$. De esta manera, no propagamos la información de que una acción ha sido negativa hasta finalizar la partida, sino que asociamos el mal resultado (perder una vida) a la acción o acciones cercanas al evento. De esta manera, el agente puede identificar antes qué comportamientos son positivos o negativos, y como consecuencia, el entrenamiento debería ser más rápido, estable, eficiente y debería encontrar estrategias de mayor calidad al tener más información concreta del entorno.

Definición

```
In [ ]: class AtariProcessorReward(AtariProcessor):
    """
    Variante de AtariProcessor que añade una penalización (life_penalty)
    cada vez que el agente pierde una vida, sumada al recorte estándar
    de la recompensa a  $[-1, 1]$ . Esto adelanta la señal de "esto es malo"
    al momento exacto en que se pierde la vida, en vez de propagarla
    solo al terminar la partida.
    """

    def __init__(self, input_shape=INPUT_SHAPE, window_length=WINDOW_LENGTH,
                 life_penalty=-1.0):
        super().__init__(input_shape, window_length)
        self.life_penalty = life_penalty
        self.current_lives = None

    def reset_lives(self, lives):
        self.current_lives = lives

    def process_reward_with_lives(self, reward, lives):
        shaped_reward = self.process_reward(reward)
        if self.current_lives is not None and lives < self.current_lives:
            shaped_reward += self.life_penalty
```

```

self.current_lives = lives
return shaped_reward

```

```

In [ ]: def train_shaped(env, agent, processor, nb_steps, log_interval=10_000):
    obs, info = env.reset(seed=SEED)
    state = processor.reset(obs)
    processor.reset_lives(info.get("lives"))

    episode_reward = 0.0
    episode_steps = 0
    episode_count = 0
    recent_rewards = deque(maxlen=20)

    start_time = time.time()
    interval_start_time = start_time
    interval_loss = []
    interval_reward = 0.0
    accumulated_q_values = []

    history = {
        "step": [], "mean_reward_20ep": [], "loss": [], "epsilon": [],
        "mean_q": [], "interval_time": [], "total_time": [],
    }

    n_intervals = nb_steps // log_interval
    pbar = tqdm(total=log_interval, desc=f"Intervalo 1/{n_intervals}", unit="step")

    for step in range(1, nb_steps + 1):
        agent.step_count = step

        if step <= agent.nb_steps_warmup:
            action = env.action_space.sample()
        else:
            action = agent.act(state, training=True)

        obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

```

```
# Recompensa moldeada: recorte + penalización por pérdida de vida
proc_reward = processor.process_reward_with_lives(reward, info.get("lives"))
next_state = processor.step(obs)

agent.remember(state, action, proc_reward, next_state, float(done))

state = next_state
episode_reward += reward
episode_steps += 1
interval_reward += proc_reward

if step > agent.nb_steps_warmup and step % agent.train_interval == 0:
    loss, q_val = agent.train_step()
    if q_val is not None:
        accumulated_q_values.append(q_val)
    if loss is not None:
        interval_loss.append(loss)

if step > agent.nb_steps_warmup and step % agent.target_model_update == 0:
    agent.update_target()

if done:
    episode_count += 1
    recent_rewards.append(episode_reward)
    obs, info = env.reset()
    state = processor.reset(obs)
    processor.reset_lives(info.get("lives"))
    episode_reward = 0.0
    episode_steps = 0

mean_r_live = np.mean(recent_rewards) if recent_rewards else 0.0
pbar.set_postfix({
    "reward": f"{mean_r_live:.1f}",
    "eps": f"{agent.policy.get_eps(step, training=True):.3f}",
})
pbar.update(1)

if step % log_interval == 0:
    now = time.time()
    elapsed_total = now - start_time
```

```

elapsed_interval = now - interval_start_time
mean_r = np.mean(recent_rewards) if recent_rewards else 0.0
mean_loss = np.mean(interval_loss) if interval_loss else 0.0
eps = agent.policy.get_eps(step, training=True)

history["step"].append(step)
history["mean_reward_20ep"].append(mean_r)
history["loss"].append(mean_loss)
history["epsilon"].append(eps)
history["interval_time"].append(elapsed_interval)
history["total_time"].append(elapsed_total)
history["mean_q"].append(np.mean(accumulated_q_values) if accumulated_q_values else 0.0)

pbar.close()
print(
    f"Step {step:>8,}/{nb_steps:,} | "
    f"reward(últ.20 ep): {mean_r:6.2f} | "
    f"loss: {mean_loss:7.4f} | "
    f"eps: {eps:5.3f} | "
    f"tiempo intervalo: {elapsed_interval:6.1f}s | "
    f"tiempo total: {elapsed_total:7.1f}s"
)
interval_loss = []
accumulated_q_values = []
interval_start_time = now

if step < nb_steps:
    current_interval = step // log_interval + 1
    pbar = tqdm(total=log_interval, desc=f"Intervalo {current_interval}/{n_intervals}", unit="step")

total_time = time.time() - start_time
print(f"\nEntrenamiento finalizado. Tiempo total: {total_time:.1f}s "
      f"({total_time/60:.2f} min, {total_time/3600:.2f} h)")

return agent, history

```

In []: NB_STEPS = 600000

```

weights_filename = f"tuned_{ENV_NAME.replace('/', '_')}_weights.pt"
history_filename = "history_tuned_SpaceInvaders.json"

```

```
print(f"Pasos de entrenamiento: {NB_STEPS:,}")
```

Pasos de entrenamiento: 600,000

Instanciación y entreno

```
In [ ]: model_tuned = DQNCNN(N_ACTIONS, WINDOW_LENGTH).to(device)

processor = AtariProcessorReward(INPUT_SHAPE, WINDOW_LENGTH, life_penalty=-1.0)

memory = ReplayMemory(capacity=1_000_000)

policy = LinearAnnealedEpsGreedy(
    value_max=1.0,
    value_min=0.1,
    value_test=0.05,
    nb_steps=NB_STEPS,
)

agent = DQNAgent(
    model=model_tuned,
    n_actions=N_ACTIONS,
    memory=memory,
    policy=policy,
    device=device,
    gamma=0.99,
    lr=1e-4,           # LR más bajo -> aprendizaje más estable
    nb_steps_warmup=NB_STEPS_WARMUP,
    target_model_update=5_000, # actualización de target más frecuente
    train_interval=4,
    batch_size=64,      # batch mayor -> gradientes menos ruidosos
)

agent, history_tuned = train_shaped(
    env,
    agent,
    processor,
    nb_steps=NB_STEPS,
)
```

```

agent.save_weights(weights_filename)
print("Pesos guardados en:", weights_filename)

with open(history_filename, "w", encoding="utf-8") as f:
    json.dump(history_tuned, f)
print("Histórico de entrenamiento guardado en:", history_filename)

```

Intervalo 1/60: 100%		10000/10000 [00:19<00:00, 506.90step/s, reward=138.2, eps=0.985]
Step 10,000/600,000	reward(últ.20 ep): 138.16 loss: 0.0000 eps: 0.985 tiempo intervalo: 19.9s tiempo total: 19.9s	1
Intervalo 2/60: 100%		10000/10000 [00:20<00:00, 518.76step/s, reward=122.2, eps=0.970]
Step 20,000/600,000	reward(últ.20 ep): 122.25 loss: 0.0000 eps: 0.970 tiempo intervalo: 20.3s tiempo total: 40.3s	4
Intervalo 3/60: 100%		10000/10000 [00:54<00:00, 188.59step/s, reward=171.0, eps=0.955]
Step 30,000/600,000	reward(últ.20 ep): 171.00 loss: 0.0122 eps: 0.955 tiempo intervalo: 54.8s tiempo total: 95.1s	9
Intervalo 4/60: 100%		10000/10000 [00:53<00:00, 191.36step/s, reward=120.5, eps=0.940]
Step 40,000/600,000	reward(últ.20 ep): 120.50 loss: 0.0113 eps: 0.940 tiempo intervalo: 53.3s tiempo total: 148.4s	14
Intervalo 5/60: 100%		10000/10000 [00:52<00:00, 191.42step/s, reward=127.5, eps=0.925]
Step 50,000/600,000	reward(últ.20 ep): 127.50 loss: 0.0107 eps: 0.925 tiempo intervalo: 52.7s tiempo total: 201.1s	20
Intervalo 6/60: 100%		10000/10000 [00:53<00:00, 189.63step/s, reward=141.5, eps=0.910]
Step 60,000/600,000	reward(últ.20 ep): 141.50 loss: 0.0107 eps: 0.910 tiempo intervalo: 53.4s tiempo total: 254.5s	25
Intervalo 7/60: 100%		10000/10000 [00:52<00:00, 188.80step/s, reward=189.2, eps=0.895]
Step 70,000/600,000	reward(últ.20 ep): 189.25 loss: 0.0111 eps: 0.895 tiempo intervalo: 52.9s tiempo total: 307.4s	30
Intervalo 8/60: 100%		10000/10000 [00:53<00:00, 189.69step/s, reward=141.8, eps=0.880]
Step 80,000/600,000	reward(últ.20 ep): 141.75 loss: 0.0113 eps: 0.880 tiempo intervalo: 53.5s tiempo total: 360.9s	36
Intervalo 9/60: 100%		10000/10000 [00:53<00:00, 189.79step/s, reward=208.5, eps=0.865]

Step 90,000/600,000 | reward(últ.20 ep): 208.50 | loss: 0.0111 | eps: 0.865 | tiempo intervalo: 53.2s | tiempo total: 41 4.1s

Intervalo 10/60: 100%  10000/10000 [00:53<00:00, 187.77step/s, reward=175.5, eps=0.850]

Step 100,000/600,000 | reward(últ.20 ep): 175.50 | loss: 0.0112 | eps: 0.850 | tiempo intervalo: 53.7s | tiempo total: 46 7.8s

Intervalo 11/60: 100%  10000/10000 [00:53<00:00, 192.02step/s, reward=171.8, eps=0.835]

Step 110,000/600,000 | reward(últ.20 ep): 171.75 | loss: 0.0112 | eps: 0.835 | tiempo intervalo: 53.5s | tiempo total: 52 1.3s

Intervalo 12/60: 100%  10000/10000 [00:53<00:00, 185.49step/s, reward=179.0, eps=0.820]

Step 120,000/600,000 | reward(últ.20 ep): 179.00 | loss: 0.0117 | eps: 0.820 | tiempo intervalo: 53.7s | tiempo total: 57 5.0s

Intervalo 13/60: 100%  10000/10000 [00:53<00:00, 184.88step/s, reward=175.8, eps=0.805]

Step 130,000/600,000 | reward(últ.20 ep): 175.75 | loss: 0.0128 | eps: 0.805 | tiempo intervalo: 54.0s | tiempo total: 62 8.9s

Intervalo 14/60: 100%  10000/10000 [00:53<00:00, 184.61step/s, reward=183.8, eps=0.790]

Step 140,000/600,000 | reward(últ.20 ep): 183.75 | loss: 0.0135 | eps: 0.790 | tiempo intervalo: 53.5s | tiempo total: 68 2.4s

Intervalo 15/60: 100%  10000/10000 [00:53<00:00, 173.85step/s, reward=177.2, eps=0.775]

Step 150,000/600,000 | reward(últ.20 ep): 177.25 | loss: 0.0131 | eps: 0.775 | tiempo intervalo: 53.9s | tiempo total: 73 6.3s

Intervalo 16/60: 100%  10000/10000 [00:53<00:00, 188.98step/s, reward=233.8, eps=0.760]

Step 160,000/600,000 | reward(últ.20 ep): 233.75 | loss: 0.0136 | eps: 0.760 | tiempo intervalo: 53.3s | tiempo total: 78 9.7s

Intervalo 17/60: 100%  10000/10000 [00:52<00:00, 180.75step/s, reward=235.0, eps=0.745]

Step 170,000/600,000 | reward(últ.20 ep): 235.00 | loss: 0.0141 | eps: 0.745 | tiempo intervalo: 52.8s | tiempo total: 84 2.5s

Intervalo 18/60: 100%  10000/10000 [00:53<00:00, 188.71step/s, reward=206.2, eps=0.730]

Step 180,000/600,000 | reward(últ.20 ep): 206.25 | loss: 0.0144 | eps: 0.730 | tiempo intervalo: 53.2s | tiempo total: 89 5.8s

Intervalo 19/60: 100%  10000/10000 [00:53<00:00, 182.74step/s, reward=188.8, eps=0.715]

Step 190,000/600,000 | reward(últ.20 ep): 188.75 | loss: 0.0146 | eps: 0.715 | tiempo intervalo: 53.6s | tiempo total: 94 9.3s

Intervalo 20/60: 100%  10000/10000 [00:53<00:00, 187.48step/s, reward=248.0, eps=0.700]
Step 200,000/600,000 | reward(últ.20 ep): 248.00 | loss: 0.0145 | eps: 0.700 | tiempo intervalo: 53.4s | tiempo total: 1002.7s

Intervalo 21/60: 100%  10000/10000 [00:53<00:00, 181.27step/s, reward=230.0, eps=0.685]
Step 210,000/600,000 | reward(últ.20 ep): 230.00 | loss: 0.0150 | eps: 0.685 | tiempo intervalo: 53.6s | tiempo total: 1056.3s

Intervalo 22/60: 100%  10000/10000 [00:53<00:00, 185.73step/s, reward=221.0, eps=0.670]
Step 220,000/600,000 | reward(últ.20 ep): 221.00 | loss: 0.0160 | eps: 0.670 | tiempo intervalo: 53.8s | tiempo total: 1110.0s

Intervalo 23/60: 100%  10000/10000 [00:53<00:00, 178.49step/s, reward=241.0, eps=0.655]
Step 230,000/600,000 | reward(últ.20 ep): 241.00 | loss: 0.0158 | eps: 0.655 | tiempo intervalo: 53.8s | tiempo total: 1163.8s

Intervalo 24/60: 100%  10000/10000 [00:53<00:00, 186.81step/s, reward=242.0, eps=0.640]
Step 240,000/600,000 | reward(últ.20 ep): 242.00 | loss: 0.0163 | eps: 0.640 | tiempo intervalo: 53.9s | tiempo total: 1217.8s

Intervalo 25/60: 100%  10000/10000 [00:54<00:00, 175.83step/s, reward=297.5, eps=0.625]
Step 250,000/600,000 | reward(últ.20 ep): 297.50 | loss: 0.0181 | eps: 0.625 | tiempo intervalo: 54.3s | tiempo total: 1272.1s

Intervalo 26/60: 100%  10000/10000 [00:54<00:00, 183.76step/s, reward=251.0, eps=0.610]
Step 260,000/600,000 | reward(últ.20 ep): 251.00 | loss: 0.0185 | eps: 0.610 | tiempo intervalo: 54.3s | tiempo total: 1326.3s

Intervalo 27/60: 100%  10000/10000 [00:53<00:00, 185.63step/s, reward=249.2, eps=0.595]
Step 270,000/600,000 | reward(últ.20 ep): 249.25 | loss: 0.0195 | eps: 0.595 | tiempo intervalo: 53.7s | tiempo total: 1380.1s

Intervalo 28/60: 100%  10000/10000 [00:53<00:00, 188.10step/s, reward=298.0, eps=0.580]
Step 280,000/600,000 | reward(últ.20 ep): 298.00 | loss: 0.0211 | eps: 0.580 | tiempo intervalo: 53.7s | tiempo total: 1433.8s

Intervalo 29/60: 100%  10000/10000 [00:54<00:00, 188.17step/s, reward=304.8, eps=0.565]
Step 290,000/600,000 | reward(últ.20 ep): 304.75 | loss: 0.0224 | eps: 0.565 | tiempo intervalo: 54.4s | tiempo total: 1488.1s

Intervalo 30/60: 100%  10000/10000 [00:53<00:00, 193.25step/s, reward=282.5, eps=0.550]

Step 300,000/600,000 | reward(últ.20 ep): 282.50 | loss: 0.0230 | eps: 0.550 | tiempo intervalo: 53.8s | tiempo total: 154 2.0s

Intervalo 31/60: 100%  10000/10000 [00:53<00:00, 186.27step/s, reward=259.5, eps=0.535]

Step 310,000/600,000 | reward(últ.20 ep): 259.50 | loss: 0.0229 | eps: 0.535 | tiempo intervalo: 53.8s | tiempo total: 159 5.8s

Intervalo 32/60: 100%  10000/10000 [00:53<00:00, 183.57step/s, reward=238.8, eps=0.520]

Step 320,000/600,000 | reward(últ.20 ep): 238.75 | loss: 0.0242 | eps: 0.520 | tiempo intervalo: 53.9s | tiempo total: 164 9.6s

Intervalo 33/60: 100%  10000/10000 [00:54<00:00, 184.71step/s, reward=297.5, eps=0.505]

Step 330,000/600,000 | reward(últ.20 ep): 297.50 | loss: 0.0257 | eps: 0.505 | tiempo intervalo: 54.1s | tiempo total: 170 3.8s

Intervalo 34/60: 100%  10000/10000 [00:53<00:00, 181.68step/s, reward=318.8, eps=0.490]

Step 340,000/600,000 | reward(últ.20 ep): 318.75 | loss: 0.0257 | eps: 0.490 | tiempo intervalo: 53.9s | tiempo total: 175 7.6s

Intervalo 35/60: 100%  10000/10000 [00:53<00:00, 185.17step/s, reward=321.8, eps=0.475]

Step 350,000/600,000 | reward(últ.20 ep): 321.75 | loss: 0.0265 | eps: 0.475 | tiempo intervalo: 53.9s | tiempo total: 181 1.5s

Intervalo 36/60: 100%  10000/10000 [00:54<00:00, 188.52step/s, reward=307.8, eps=0.460]

Step 360,000/600,000 | reward(últ.20 ep): 307.75 | loss: 0.0276 | eps: 0.460 | tiempo intervalo: 54.2s | tiempo total: 186 5.8s

Intervalo 37/60: 100%  10000/10000 [00:54<00:00, 180.78step/s, reward=328.2, eps=0.445]

Step 370,000/600,000 | reward(últ.20 ep): 328.25 | loss: 0.0303 | eps: 0.445 | tiempo intervalo: 54.3s | tiempo total: 192 0.0s

Intervalo 38/60: 100%  10000/10000 [00:54<00:00, 182.47step/s, reward=286.8, eps=0.430]

Step 380,000/600,000 | reward(últ.20 ep): 286.75 | loss: 0.0307 | eps: 0.430 | tiempo intervalo: 54.2s | tiempo total: 197 4.2s

Intervalo 39/60: 100%  10000/10000 [00:53<00:00, 186.40step/s, reward=317.2, eps=0.415]

Step 390,000/600,000 | reward(últ.20 ep): 317.25 | loss: 0.0311 | eps: 0.415 | tiempo intervalo: 54.0s | tiempo total: 202 8.2s

Intervalo 40/60: 100%  10000/10000 [00:54<00:00, 188.80step/s, reward=288.5, eps=0.400]

Step 400,000/600,000 | reward(últ.20 ep): 288.50 | loss: 0.0314 | eps: 0.400 | tiempo intervalo: 54.1s | tiempo total: 208 2.4s

Intervalo 41/60: 100%  10000/10000 [00:54<00:00, 182.50step/s, reward=307.8, eps=0.385]
Step 410,000/600,000 | reward(últ.20 ep): 307.75 | loss: 0.0318 | eps: 0.385 | tiempo intervalo: 54.3s | tiempo total: 213 6.6s

Intervalo 42/60: 100%  10000/10000 [00:54<00:00, 185.91step/s, reward=291.2, eps=0.370]
Step 420,000/600,000 | reward(últ.20 ep): 291.25 | loss: 0.0337 | eps: 0.370 | tiempo intervalo: 54.2s | tiempo total: 219 0.8s

Intervalo 43/60: 100%  10000/10000 [00:55<00:00, 178.79step/s, reward=285.8, eps=0.355]
Step 430,000/600,000 | reward(últ.20 ep): 285.75 | loss: 0.0346 | eps: 0.355 | tiempo intervalo: 55.5s | tiempo total: 224 6.3s

Intervalo 44/60: 100%  10000/10000 [00:54<00:00, 187.97step/s, reward=299.2, eps=0.340]
Step 440,000/600,000 | reward(últ.20 ep): 299.25 | loss: 0.0346 | eps: 0.340 | tiempo intervalo: 54.5s | tiempo total: 230 0.7s

Intervalo 45/60: 100%  10000/10000 [00:54<00:00, 190.87step/s, reward=318.0, eps=0.325]
Step 450,000/600,000 | reward(últ.20 ep): 318.00 | loss: 0.0347 | eps: 0.325 | tiempo intervalo: 54.2s | tiempo total: 235 4.9s

Intervalo 46/60: 100%  10000/10000 [00:54<00:00, 185.62step/s, reward=359.8, eps=0.310]
Step 460,000/600,000 | reward(últ.20 ep): 359.75 | loss: 0.0346 | eps: 0.310 | tiempo intervalo: 54.1s | tiempo total: 240 9.0s

Intervalo 47/60: 100%  10000/10000 [00:54<00:00, 187.39step/s, reward=331.8, eps=0.295]
Step 470,000/600,000 | reward(últ.20 ep): 331.75 | loss: 0.0354 | eps: 0.295 | tiempo intervalo: 54.2s | tiempo total: 246 3.2s

Intervalo 48/60: 100%  10000/10000 [00:55<00:00, 176.72step/s, reward=323.8, eps=0.280]
Step 480,000/600,000 | reward(últ.20 ep): 323.75 | loss: 0.0354 | eps: 0.280 | tiempo intervalo: 55.0s | tiempo total: 251 8.2s

Intervalo 49/60: 100%  10000/10000 [00:54<00:00, 172.07step/s, reward=336.0, eps=0.265]
Step 490,000/600,000 | reward(últ.20 ep): 336.00 | loss: 0.0344 | eps: 0.265 | tiempo intervalo: 54.7s | tiempo total: 257 2.9s

Intervalo 50/60: 100%  10000/10000 [00:57<00:00, 182.74step/s, reward=299.8, eps=0.250]
Step 500,000/600,000 | reward(últ.20 ep): 299.75 | loss: 0.0358 | eps: 0.250 | tiempo intervalo: 57.7s | tiempo total: 263 0.6s

Intervalo 51/60: 100%  10000/10000 [00:54<00:00, 187.23step/s, reward=307.8, eps=0.235]

Step 510,000/600,000 | reward(últ.20 ep): 307.75 | loss: 0.0377 | eps: 0.235 | tiempo intervalo: 54.7s | tiempo total: 268 5.3s

Intervalo 52/60: 100%  10000/10000 [00:54<00:00, 191.27step/s, reward=367.5, eps=0.220]

Step 520,000/600,000 | reward(últ.20 ep): 367.50 | loss: 0.0379 | eps: 0.220 | tiempo intervalo: 54.6s | tiempo total: 273 9.9s

Intervalo 53/60: 100%  10000/10000 [00:54<00:00, 186.16step/s, reward=372.2, eps=0.205]

Step 530,000/600,000 | reward(últ.20 ep): 372.25 | loss: 0.0366 | eps: 0.205 | tiempo intervalo: 54.6s | tiempo total: 279 4.5s

Intervalo 54/60: 100%  10000/10000 [00:54<00:00, 178.03step/s, reward=350.0, eps=0.190]

Step 540,000/600,000 | reward(últ.20 ep): 350.00 | loss: 0.0368 | eps: 0.190 | tiempo intervalo: 54.6s | tiempo total: 284 9.1s

Intervalo 55/60: 100%  10000/10000 [00:54<00:00, 185.10step/s, reward=353.8, eps=0.175]

Step 550,000/600,000 | reward(últ.20 ep): 353.75 | loss: 0.0380 | eps: 0.175 | tiempo intervalo: 54.5s | tiempo total: 290 3.6s

Intervalo 56/60: 100%  10000/10000 [00:54<00:00, 180.52step/s, reward=335.2, eps=0.160]

Step 560,000/600,000 | reward(últ.20 ep): 335.25 | loss: 0.0380 | eps: 0.160 | tiempo intervalo: 54.7s | tiempo total: 295 8.3s

Intervalo 57/60: 100%  10000/10000 [00:54<00:00, 182.42step/s, reward=349.0, eps=0.145]

Step 570,000/600,000 | reward(últ.20 ep): 349.00 | loss: 0.0376 | eps: 0.145 | tiempo intervalo: 54.8s | tiempo total: 301 3.1s

Intervalo 58/60: 100%  10000/10000 [00:55<00:00, 179.45step/s, reward=352.8, eps=0.130]

Step 580,000/600,000 | reward(últ.20 ep): 352.75 | loss: 0.0367 | eps: 0.130 | tiempo intervalo: 55.8s | tiempo total: 306 9.0s

Intervalo 59/60: 100%  10000/10000 [00:56<00:00, 175.04step/s, reward=337.8, eps=0.115]

Step 590,000/600,000 | reward(últ.20 ep): 337.75 | loss: 0.0379 | eps: 0.115 | tiempo intervalo: 56.8s | tiempo total: 312 5.8s

Intervalo 60/60: 100%  10000/10000 [00:57<00:00, 172.90step/s, reward=363.8, eps=0.100]

Step 600,000/600,000 | reward(últ.20 ep): 363.75 | loss: 0.0381 | eps: 0.100 | tiempo intervalo: 57.6s | tiempo total: 318 3.3s

Entrenamiento finalizado. Tiempo total: 3183.3s (53.06 min, 0.88 h)

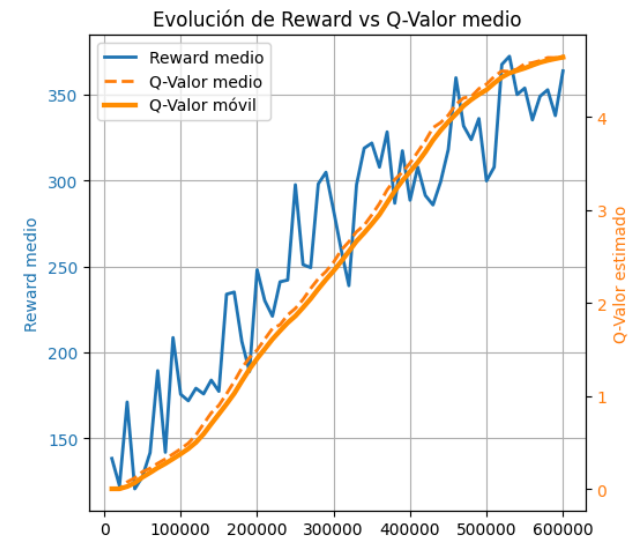
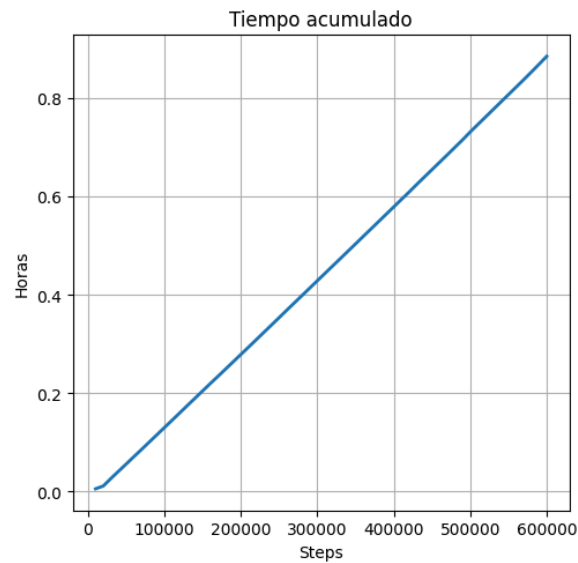
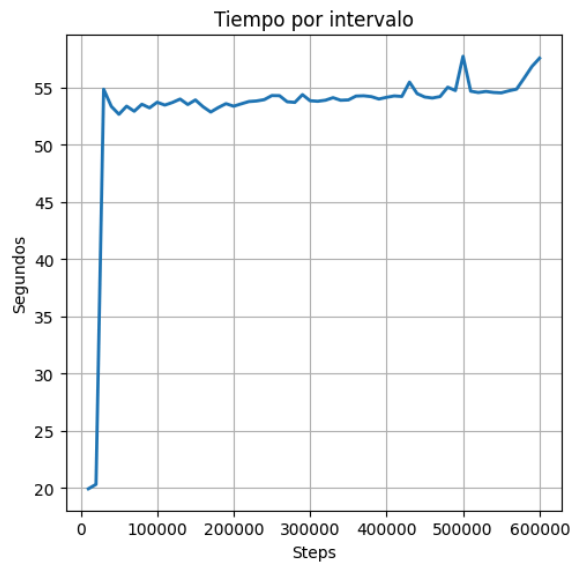
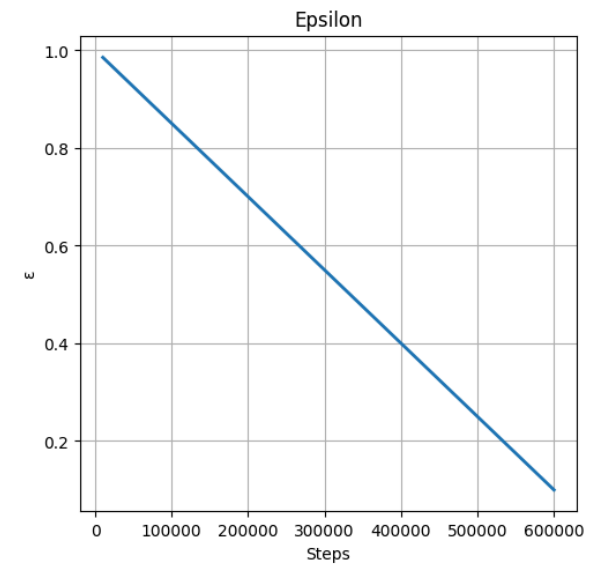
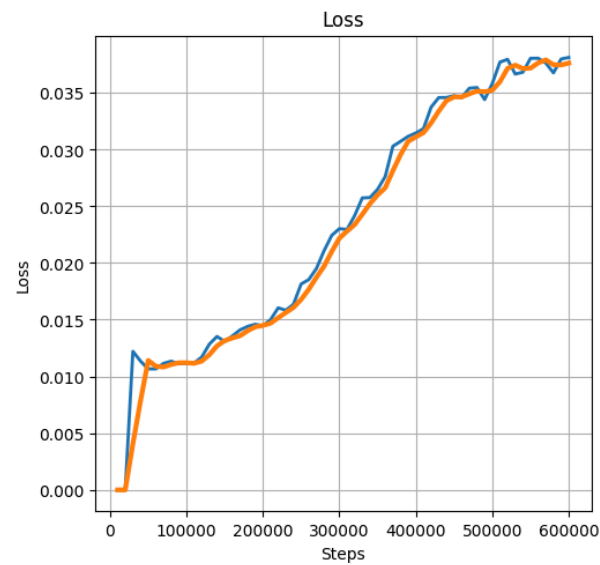
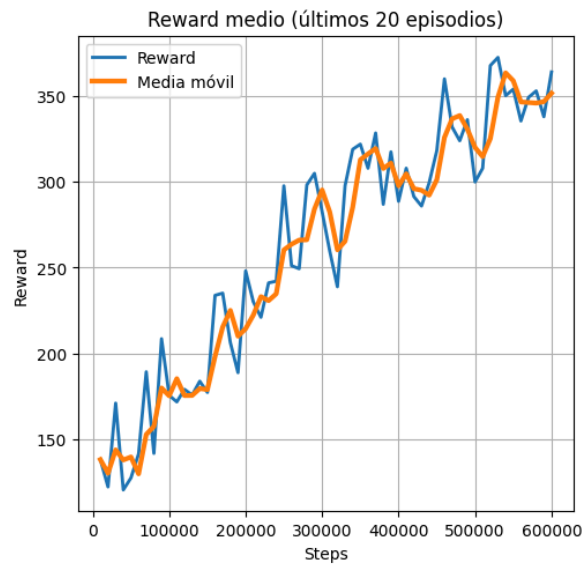
Pesos guardados en: tuned_ALE_SpaceInvaders-v5_weights.pt

Histórico de entrenamiento guardado en: history_tuned_SpaceInvaders.json

Resultados

```
In [ ]: with open("history_tuned_SpaceInvaders.json", "r", encoding="utf-8") as f:
        history = json.load(f)

        plot_training_history(history)
```



3.4 - Análisis, comparación de resultados y conclusiones

```
In [ ]: # =====
# COMPARATIVA COMPLETA: carga de modelos, test y gráficas de justificación
```

```
# =====

import os
import json
import numpy as np
import matplotlib.pyplot as plt

N_TEST_EPISODES = 20 # nº de episodios de test por modelo

MODELS_TO_COMPARE = [
    {
        "name": "DQN",
        "agent_class": DQNAgent,
        "model_class": DQNCNN,
        "weights_file": f"dqn_{ENV_NAME.replace('/', '_')}_weights.pt",
        "history_file": "history_dqn_SpaceInvaders.json",
    },
    {
        "name": "DDQN",
        "agent_class": DDQNAgent,
        "model_class": DQNCNN,
        "weights_file": f"ddqn_{ENV_NAME.replace('/', '_')}_weights.pt",
        "history_file": "history_ddqn_SpaceInvaders.json",
    },
    {
        "name": "Dueling DQN",
        "agent_class": DuelingDQNAgent,
        "model_class": DuelingDQNCNN,
        "weights_file": f"dueling_{ENV_NAME.replace('/', '_')}_weights.pt",
        "history_file": "history_dueling_SpaceInvaders.json",
    },
    {
        "name": "DQN Tuned",
        "agent_class": DQNAgent,
        "model_class": DQNCNN,
        "weights_file": f"tuned_{ENV_NAME.replace('/', '_')}_weights.pt",
        "history_file": "history_tuned_SpaceInvaders.json",
    },
]
```

```

def build_agent_for_test(agent_class, model_class):
    """Crea un agente listo para test (memoria mínima, epsilon de test fijo)."""
    model = model_class(N_ACTIONS, WINDOW_LENGTH).to(device)
    memory = ReplayMemory(capacity=1000)
    policy = LinearAnnealedEpsGreedy(
        value_max=1.0, value_min=0.1, value_test=0.05, nb_steps=1
    )
    return agent_class(
        model=model,
        n_actions=N_ACTIONS,
        memory=memory,
        policy=policy,
        device=device,
    )

# -----
# 1. Carga de modelos + test + carga de históricos
# -----

results = {}
histories = {}

for cfg in MODELS_TO_COMPARE:
    name = cfg["name"]

    # --- Test con pesos entrenados ---
    if os.path.exists(cfg["weights_file"]):
        print(f"\n{' '*15} Evaluando: {name} {' '*15}")
        agent = build_agent_for_test(cfg["agent_class"], cfg["model_class"])
        agent.load_weights(cfg["weights_file"])

        processor = AtariProcessor(INPUT_SHAPE, WINDOW_LENGTH)
        rewards, mean_reward = test(
            agent, processor, ENV_NAME, n_episodes=N_TEST_EPISODES, render=False
        )
        results[name] = {
            "rewards": rewards,
            "mean_reward": mean_reward,
            "std_reward": float(np.std(rewards)),
        }
    else:

```

```

        print(f"[AVISO] No se encontraron pesos para '{name}' en {cfg['weights_file']}. Se omite del test.")

# --- Histórico de entrenamiento ---
if os.path.exists(cfg["history_file"]):
    with open(cfg["history_file"], "r", encoding="utf-8") as f:
        histories[name] = json.load(f)
else:
    print(f"[AVISO] No se encontró el histórico de '{name}' en {cfg['history_file']}. Se omite de las curvas.")

# -----
# 2. Panel de gráficas de justificación
# -----
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# (a) Curva de aprendizaje: reward medio vs steps
for name, h in histories.items():
    axes[0, 0].plot(h["step"], h["mean_reward_20ep"], linewidth=2, label=name)
axes[0, 0].set_title("Evolución del reward durante el entrenamiento")
axes[0, 0].set_xlabel("Steps")
axes[0, 0].set_ylabel("Reward medio (últ. 20 ep)")
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# (b) Q-valor medio vs steps (detecta sobreestimación DQN vs DDQN)
for name, h in histories.items():
    if "mean_q" in h and len(h["mean_q"]) > 0:
        axes[0, 1].plot(h["step"], h["mean_q"], linewidth=2, label=name)
axes[0, 1].set_title("Evolución del Q-valor medio estimado")
axes[0, 1].set_xlabel("Steps")
axes[0, 1].set_ylabel("Q-valor")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# (c) Recompensa media en test, +/- desviación típica
if results:
    names_r = list(results.keys())
    means = [results[n]["mean_reward"] for n in names_r]
    stds = [results[n]["std_reward"] for n in names_r]

    axes[1, 0].bar(names_r, means, yerr=stds, capsize=6, color="tab:blue", alpha=0.8)
    axes[1, 0].set_title(f"Recompensa media en test ({N_TEST_EPISODES} episodios)")

```



```
axes[1, 0].set_ylabel("Recompensa media")
axes[1, 0].grid(True, axis="y", alpha=0.3)
for i, m in enumerate(means):
    axes[1, 0].text(i, m + max(means) * 0.02, f"{m:.1f}", ha="center")

# (d) Distribución de recompensas por episodio (boxplot)
box_data = [results[n]["rewards"] for n in names_r]
axes[1, 1].boxplot(box_data)
axes[1, 1].set_xticks(range(1, len(names_r) + 1))
axes[1, 1].set_xticklabels(names_r)
axes[1, 1].set_title("Distribución de recompensas por episodio")
axes[1, 1].set_ylabel("Recompensa por episodio")
axes[1, 1].grid(True, axis="y", alpha=0.3)
else:
    axes[1, 0].axis("off")
    axes[1, 1].axis("off")
    print("No se ha podido evaluar ningún modelo en test: revisa los ficheros de pesos.")

plt.tight_layout()
plt.savefig("comparativa_justificacion_modelos.png", dpi=300, bbox_inches="tight")
plt.show()
```

===== Evaluando: DQN =====

Episodio 1: reward: 595.000, steps: 729
Episodio 2: reward: 440.000, steps: 762
Episodio 3: reward: 390.000, steps: 647
Episodio 4: reward: 340.000, steps: 618
Episodio 5: reward: 315.000, steps: 592
Episodio 6: reward: 470.000, steps: 693
Episodio 7: reward: 480.000, steps: 805
Episodio 8: reward: 365.000, steps: 661
Episodio 9: reward: 460.000, steps: 866
Episodio 10: reward: 470.000, steps: 907
Episodio 11: reward: 395.000, steps: 644
Episodio 12: reward: 600.000, steps: 974
Episodio 13: reward: 410.000, steps: 804
Episodio 14: reward: 285.000, steps: 644
Episodio 15: reward: 330.000, steps: 615
Episodio 16: reward: 545.000, steps: 579
Episodio 17: reward: 275.000, steps: 459
Episodio 18: reward: 420.000, steps: 838
Episodio 19: reward: 295.000, steps: 512
Episodio 20: reward: 295.000, steps: 580

Recompensa media en 20 episodios: 408.750

===== Evaluando: DDQN =====

Episodio 1: reward: 520.000, steps: 960
Episodio 2: reward: 290.000, steps: 569
Episodio 3: reward: 380.000, steps: 570
Episodio 4: reward: 400.000, steps: 629
Episodio 5: reward: 570.000, steps: 1049
Episodio 6: reward: 225.000, steps: 423
Episodio 7: reward: 385.000, steps: 577
Episodio 8: reward: 570.000, steps: 830
Episodio 9: reward: 325.000, steps: 518
Episodio 10: reward: 570.000, steps: 870
Episodio 11: reward: 245.000, steps: 519
Episodio 12: reward: 410.000, steps: 725
Episodio 13: reward: 115.000, steps: 364
Episodio 14: reward: 275.000, steps: 609
Episodio 15: reward: 310.000, steps: 486
Episodio 16: reward: 440.000, steps: 762

Episodio 17: reward: 525.000, steps: 749
Episodio 18: reward: 325.000, steps: 545
Episodio 19: reward: 170.000, steps: 380
Episodio 20: reward: 380.000, steps: 609

Recompensa media en 20 episodios: 371.500

===== Evaluando: Dueling DQN =====

Episodio 1: reward: 320.000, steps: 541
Episodio 2: reward: 305.000, steps: 419
Episodio 3: reward: 390.000, steps: 671
Episodio 4: reward: 405.000, steps: 599
Episodio 5: reward: 295.000, steps: 503
Episodio 6: reward: 395.000, steps: 593
Episodio 7: reward: 540.000, steps: 969
Episodio 8: reward: 370.000, steps: 510
Episodio 9: reward: 470.000, steps: 704
Episodio 10: reward: 685.000, steps: 770
Episodio 11: reward: 465.000, steps: 956
Episodio 12: reward: 570.000, steps: 807
Episodio 13: reward: 570.000, steps: 933
Episodio 14: reward: 370.000, steps: 589
Episodio 15: reward: 470.000, steps: 752
Episodio 16: reward: 545.000, steps: 1186
Episodio 17: reward: 415.000, steps: 781
Episodio 18: reward: 335.000, steps: 557
Episodio 19: reward: 495.000, steps: 790
Episodio 20: reward: 395.000, steps: 659

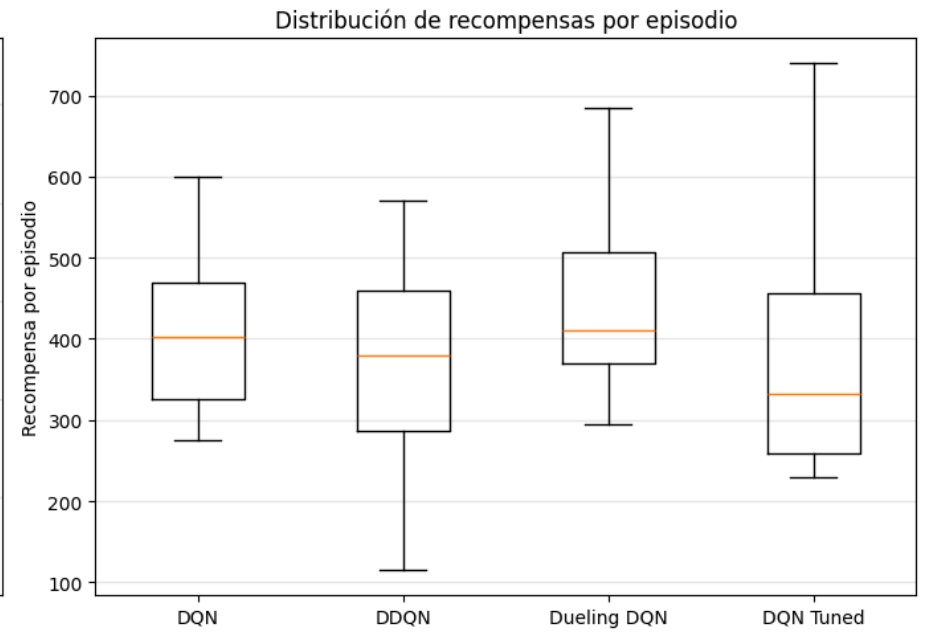
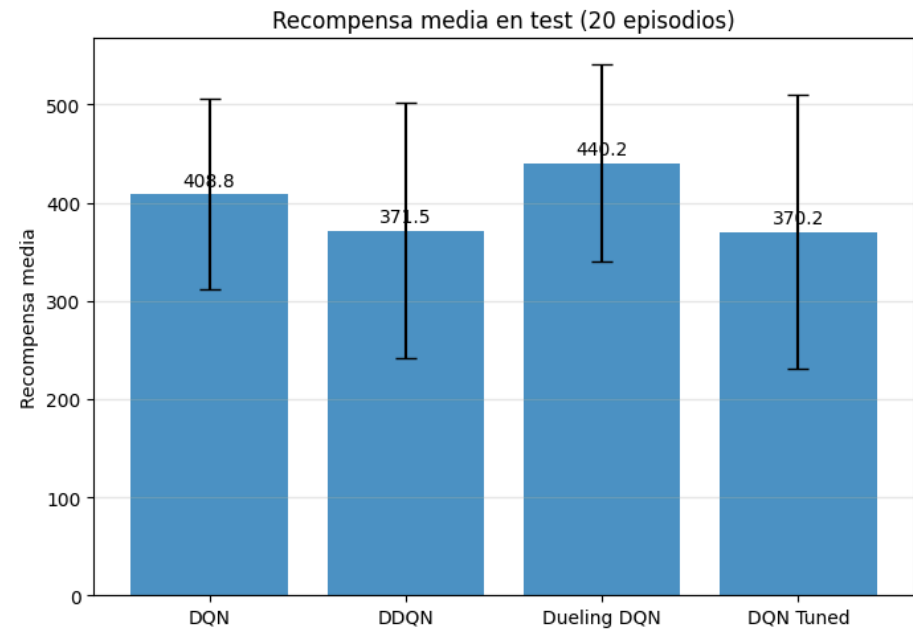
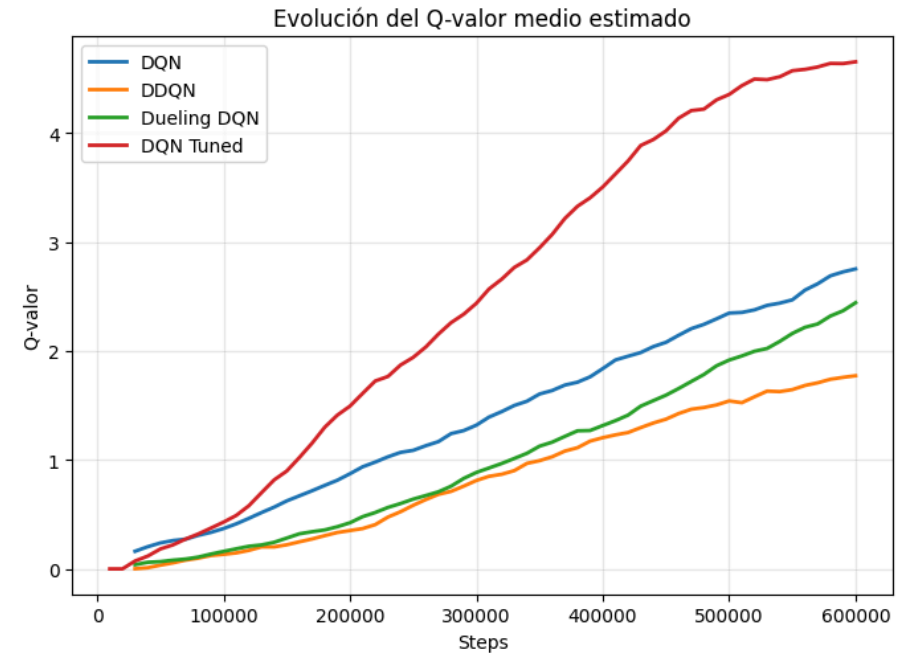
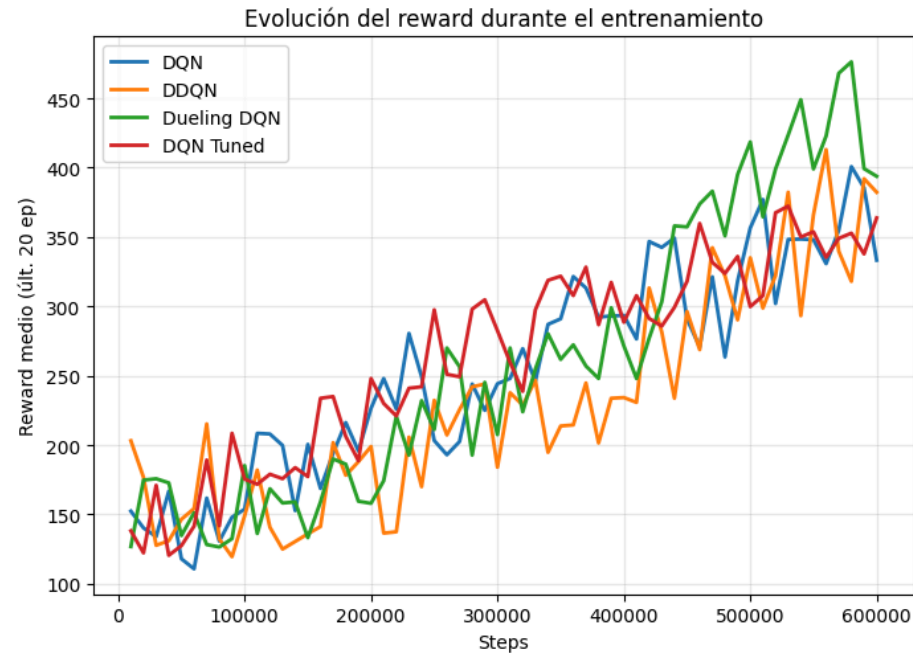
Recompensa media en 20 episodios: 440.250

===== Evaluando: DQN Tuned =====

Episodio 1: reward: 240.000, steps: 414
Episodio 2: reward: 245.000, steps: 541
Episodio 3: reward: 445.000, steps: 616
Episodio 4: reward: 365.000, steps: 616
Episodio 5: reward: 500.000, steps: 702
Episodio 6: reward: 290.000, steps: 453
Episodio 7: reward: 275.000, steps: 551
Episodio 8: reward: 490.000, steps: 946
Episodio 9: reward: 230.000, steps: 488

Episodio 10: reward: 315.000, steps: 611
Episodio 11: reward: 255.000, steps: 525
Episodio 12: reward: 640.000, steps: 877
Episodio 13: reward: 365.000, steps: 700
Episodio 14: reward: 260.000, steps: 438
Episodio 15: reward: 395.000, steps: 733
Episodio 16: reward: 350.000, steps: 673
Episodio 17: reward: 740.000, steps: 1033
Episodio 18: reward: 240.000, steps: 510
Episodio 19: reward: 500.000, steps: 945
Episodio 20: reward: 265.000, steps: 547

Recompensa media en 20 episodios: 370.250



3.4.1 - Resultados de entrenamiento

DQN

El DQN mantiene un aprendizaje estable, alcanzando recompensas finales en torno a **350–390**, con picos cercanos a **400**. El Q-valor medio crece hasta aproximadamente **2.8**, reflejando la conocida tendencia del DQN a sobreestimar los valores de acción.

Double DQN

El DDQN presenta un entrenamiento más irregular, aunque termina alrededor de **380–390** puntos. El Q-valor medio final es el más bajo (≈ 1.8), confirmando que reduce la sobreestimación respecto al DQN.

Dueling DQN

El Dueling DQN obtiene la mejor evolución durante el entrenamiento, alcanzando recompensas próximas a **400** con un pico cercano a **480**. Su Q-valor medio final ronda **2.4–2.5**, situándose entre DQN y DDQN.

DQN Tuned

El DQN Tuned muestra un entrenamiento muy estable, finalizando alrededor de **360–390** puntos. Sin embargo, el Q-valor medio crece hasta ≈ 4.7 , muy por encima del resto, indicando una fuerte sobreestimación pese a su buen rendimiento.

3.4.2 - Resultados de test (20 episodios, $\epsilon = 0.05$)

Modelo	Recompensa media
Dueling DQN	440.2
DQN	408.8
DDQN	371.5
DQN Tuned	370.2

Los diagramas de caja muestran además que el Dueling DQN posee la mediana más alta y una distribución desplazada hacia recompensas superiores, aunque con cierta variabilidad. El DQN mantiene un comportamiento sólido y consistente. DDQN y DQN Tuned presentan medias

similares, siendo el Tuned el de mayor dispersión, con episodios muy buenos pero también otros claramente inferiores.

3.4.3 - Discusión

El **Dueling DQN** es el mejor modelo tanto durante el entrenamiento como en la evaluación final, alcanzando la mayor recompensa media (**440.2**).

El **DQN** es una línea base muy competitiva (**408.8**), ofreciendo un equilibrio entre estabilidad y rendimiento.

El **DDQN** confirma su principal ventaja teórica: mantiene los Q-valores más bajos y reduce la sobreestimación. No obstante, esa mejora no se traduce en el mejor rendimiento, quedando por detrás de DQN y Dueling.

El **DQN Tuned** consigue un rendimiento similar al DDQN en test (**370.2**), pero destaca por el crecimiento excesivo del Q-valor estimado (≈ 4.7), señal de una importante sobreestimación. Aun así, esta sobreestimación no se traduce en una mejora del rendimiento final.

3.4.4 - Conclusiones

1. Los cuatro modelos aprenden correctamente a jugar a Space Invaders y superan ampliamente un agente aleatorio.
2. **Dueling DQN** es el mejor modelo global, liderando tanto el entrenamiento como la evaluación final.
3. **DQN** continúa siendo una referencia muy sólida, con un rendimiento cercano al mejor modelo.
4. **DDQN** reduce eficazmente la sobreestimación del Q-valor, aunque ello no implica obtener la mayor recompensa.
5. **DQN Tuned** alcanza un rendimiento competitivo, pero presenta la mayor sobreestimación de todos los modelos.
6. Los resultados muestran que un mayor Q-valor estimado no implica necesariamente una mejor política, siendo imprescindible evaluar siempre mediante episodios de test.

3.4.5 - Líneas de trabajo futuro

- Evaluar con múltiples semillas.
- Aumentar el entrenamiento hasta 1–2 millones de pasos.
- Combinar **Dueling + Double DQN** para aprovechar simultáneamente ambas mejoras.
- Ajustar los hiperparámetros del modelo Tuned para controlar la sobreestimación del Q-valor.
- Incorporar evaluación periódica durante el entrenamiento para detectar divergencias entre aprendizaje y rendimiento real.

In []: